TUGAS BESAR 1

MATA KULIAH DASAR INTELEGENSI ARTIFISIAL IF3070



Penerapan Hill-Climbing, Simulated Annealing, dan Genetic Algorithm untuk Memaksimalkan Shipping Fee pada Penataan Muatan Kapal

Disusun Oleh

Kelompok 29 “dai dai dai”

18224010 Gregory William Sutjipto

18224062 Johanes Sebastian Siddhadeza

18224066 Emir Muhammad Firassiyan

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

INSTITUT TEKNOLOGI BANDUNG

2026

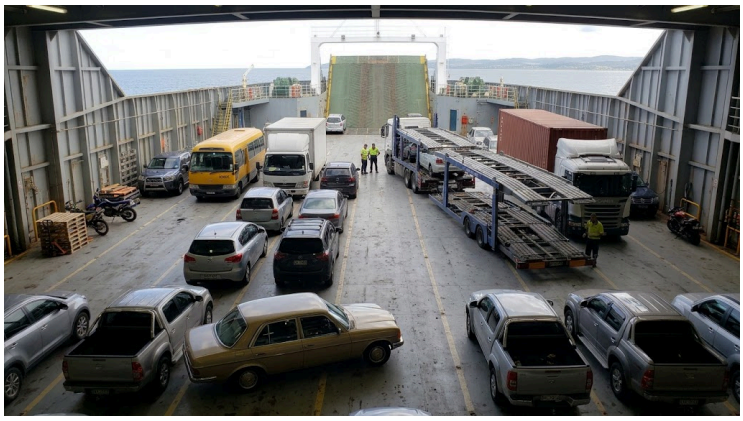
DAFTAR ISI

DAFTAR ISI **2**
BAB 1 - PENDAHULUAN **4**
1.1. Latar Belakang Persoalan 4
1.2. Entitas dan Variabel Persoalan 5
1.3. Tujuan dan Batasan 6
1.4. Asumsi dan Cakupan Persoalan 7
BAB 2 - PEMBAHASAN **8**
2.1. Rancangan Implementasi Umum 8
2.1.1. Representasi State 8
2.1.2. Penanganan Batasan 9
2.1.3. Fungsi Objective 10
2.1.4. State Awal dan Langkah Neighbor 10
2.1.5. Data Uji dan Pengaturan Eksperimen 11
2.1.6. Visualisasi 12
2.2. Hill Climbing 13
2.2.1. Cara Kerja Umum 13
2.2.2. Source Code 14
2.2.3. Hasil Eksperimen Hill Climbing - Standar 18
2.2.4. Varian Hill Climbing (Implementasi Bonus) 20
2.2.5. Hasil Eksperimen Varians Hill Climbing 24
2.2.6. Pembahasan Hasil 29
2.3. Simulated Annealing 31
2.3.1. Cara Kerja Umum 31
2.3.2. Source Code 33
2.3.3. Hasil Eksperimen 36
2.3.4. Pembahasan Hasil 41
2.4. Genetic Algorithm 42
2.4.1. Cara Kerja Umum 42
2.4.2. Source Code 43
2.4.3. Hasil Eksperimen 49
2.4.4. Pembahasan Hasil 58
2.5. Analisis Perbandingan 59
2.5.1. Ringkasan Lintas Algoritma 59
2.5.2. Kedekatan ke Global Optimum 60
2.5.3. Perbandingan Hasil antar Algoritma 60
2.5.4. Pebandingan Durasi 61
2.5.5. Konsistensi Hasil 62
2.5.6. Pengaruh Objective Berbobot ETA 63
BAB 3 - KESIMPULAN DAN SARAN **65**

3.1. Kesimpulan..... 65
3.2. Saran..... 66
BAB 4 - PEMBAGIAN TUGAS.....67
BAB 5 - REFERENSI..... 68
BAB 6 - LAMPIRAN..... 69

BAB 1 - PENDAHULUAN

1.1. Latar Belakang Persoalan



Gambar 1.1.1 Kondisi As-is geladak kapal

Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Akibatnya, ruang geladak sering tidak termanfaatkan dengan baik, dan keputusan memilih kendaraan mana yang diangkut belum tentu menghasilkan keuntungan yang optimal. Persoalan ini tidak sederhana karena geladak memiliki ruang yang terbatas dan kapal memiliki kapasitas muatan maksimum, sehingga tidak semua kendaraan dapat diangkut sekaligus. Kendaraan yang tidak diangkut tetap berada di luar kapal. Seperti pada gambar diatas, dapat dilihat penyusunan yang tidak optimal menyebabkan banyak ruang terbuang yang sebenarnya masih bisa digunakan untuk kendaraan lain parkir.



Gambar 1.1.2 Kondisi to-be geladak kapal

Berbeda dengan gambar diatas, dimana seluruh mobil berada di dalam posisi yang optimal sehingga lahan parkir digunakan secara menyeluruh. Untuk meraih state tersebut, dibutuhkan sebuah sistem yang mendeteksi seluruh keputusan penempatan kendaraan. Sistem tersebut harus menentukan kendaraan mana yang diangkut serta posisi dan

orientasi setiap kendaraan di geladak, mengevaluasi berbagai kemungkinan konfigurasi, dan menghasilkan penataan dengan total tarif pengiriman (shipping fee) yang sebesar mungkin. Jumlah kemungkinan konfigurasinya sangat besar, karena setiap kendaraan dapat diangkut atau tidak, dapat diletakkan di banyak posisi, dan dapat diputar. Mencoba semua kemungkinan secara langsung tidak praktis, sehingga diperlukan pendekatan pencarian yang sistematis.

1.2. Entitas dan Variabel Persoalan

Persoalan ini melibatkan dua entitas, yaitu kendaraan (Vehicle) dan kapal (Ship). Kendaraan dan geladak kapal sama-sama dimodelkan sebagai bidang dua dimensi berbentuk persegi atau persegi panjang yang tersusun dari petak-petak (grid), sehingga semua ukuran dinyatakan dalam satuan petak.

Adapula variabel pada entitas pertama (*vehicle)* adalah sebagai berikut

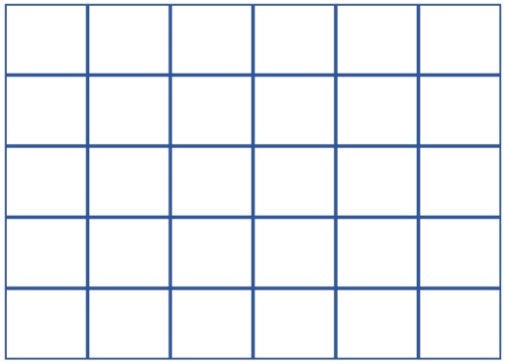
Tabel 1.2.1 Variabel Entitas Vehicle

Atribut	Contoh	Keterangan
id	ITB	Pengenal unik kendaraan
Dimensions	{w:3, l:4}	Ukuran kendaraan, terdiri atas lebar (*width*) dan panjang (*length*), dalam satuan petak
Orientation	Horizontal	Arah kendaraan diletakkan, yaitu *Horizontal* atau *Vertical*
ShippingFee	60	Tarif atau keuntungan pengiriman kendaraan, dalam satuan juta rupiah
Weight	5	Berat kendaraan, dalam satuan unit
ETA	3	*Estimated Time of Arrival*, perkiraan waktu kedatangan kendaraan, dalam satuan sisa hari

Selain variabel *vehicle*, variabel kedua yaitu kapal juga memiliki atribut tersendiri

Tabel 1.2.2 Variabel Entitas Kapal

Atribut	Contoh	Keterangan
Dimensions	{w:10, l:20}	Ukuran geladak kapal, terdiri atas lebar (*width*) dan panjang (*length*), dalam satuan petak
MaxCapacity	500	Kapasitas angkut maksimum kapal, dalam satuan unit (sama dengan satuan berat kendaraan)



Gambar 1.2.1 Peta geladak dan analogi dimensi kendaraan

Pada bagian dimensions, orientasi dapat dibayangkan sebagai peta yang memiliki *grid* di dalamnya. Sehingga besarnya sebuah kendaraan akan mengambil porsi sebanyak beberapa kotak di dalam *grid*, dimana keseluruhan *grid* dianggap sebagai seluruh geladak kapal.

1.3. Tujuan dan Batasan

Persoalan ini bertujuan memaksimalkan total tarif pengiriman (ShippingFee) dari kendaraan yang diangkut kapal. Solusinya berupa keputusan kendaraan mana yang diangkut, serta posisi dan orientasi setiap kendaraan yang diangkut di geladak. Maka dari itu, domain dapat ditentukan sebagai berikut: jika ada n kendaraan dan kendaraan ke-i bernilai fee_i, serta $x_i = 1$ bila kendaraan itu diangkut dan $x_i = 0$ bila tidak, maka tujuannya adalah untuk mencari:

$$\Sigma\,(x_i \cdot fee_i) \quad \text{maksimal}$$

Dalam pencarian $\Sigma\ (x_i \cdot fee_i)$ maksimal, terdapat beberapa batasan solusi yang tidak boleh dilanggar, antara lain:

a. Tidak saling bertumpuk (no overlapping). Tidak boleh ada satu petak geladak yang ditempati lebih dari satu kendaraan.
b. Berada di dalam batas geladak. Seluruh petak yang ditempati sebuah kendaraan harus berada di dalam dimensi geladak kapal.
c. Tidak melebihi kapasitas. Total berat kendaraan yang diangkut tidak boleh melebihi kapasitas maksimum kapal sehingga diperoleh

$$\Sigma\ (x_i \cdot weight_i) \leq MaxCapacity.$$

Kendaraan yang tidak diangkut berada seluruhnya di luar kapal, tidak menempati petak geladak, dan tidak menyumbang nilai maupun berat.

1.4. Asumsi dan Cakupan Persoalan

Agar persoalan dapat didefinisikan secara jelas, terdapat beberapa asumsi yang digunakan lain sebagai berikut:

1. Geladak berbentuk grid. Geladak dibagi menjadi petak-petak berukuran sama, dan setiap kendaraan menempati sejumlah petak utuh. Kendaraan tidak boleh diletakkan di antara dua petak.
2. Kendaraan berbentuk persegi panjang. Setiap kendaraan menempati blok petak dengan ukuran tetap (lebar × panjang) dan hanya boleh berorientasi horizontal atau vertikal, tidak miring.
3. Data kendaraan dan kapal sudah diketahui dan tetap. Ukuran, berat, dan *ShippingFee* setiap kendaraan, serta dimensi dan kapasitas kapal, diberikan sebagai masukan dan tidak berubah selama pencarian.
4. Hanya ada satu kapal dan satu geladak. Semua kendaraan bersaing untuk ruang dan kapasitas di kapal yang sama.
5. Setiap kendaraan paling banyak diangkut satu kali. Satu kendaraan tidak boleh diletakkan di dua tempat sekaligus.
6. Hanya tiga batasan yang diperhitungkan. Yang berlaku hanyalah tidak bertumpuk, tetap di dalam geladak, dan tidak melebihi kapasitas berat. Faktor lain tidak

diperhitungkan, misalnya keseimbangan kapal, urutan naik-turun kendaraan, jalur akses, dan jarak antar kendaraan

Dimana masalah utama yang berusaha diselesaikan untuk memaksimalkan total Shipping Fee. Di lain sisi, tujuan alternatif (memaksimalkan ETA) juga diperhitungkan dalam proyek kali ini.

BAB 2 - PEMBAHASAN

2.1. Rancangan Implementasi Umum

Terdapat beberapa rancangan yang digunakan secara bersama oleh ketiga algoritma seperti cara penyimpanan solusi, penanganan terhadap batasan, dan perhitungan terhadap nilai solusi awal dan solusi *neighbor* untuk menjaga keadilan dalam melakukan perbandingan dari satu algoritma ke algoritma lainnya.

2.1.1. Representasi State

Dalam hal ini, state merupakan keputusan untuk semua kendaraan, yaitu diangkut atau tidak, dan jika diangkut, di mana dan menghadap ke mana. Algoritma pencarian bekerja dengan berpindah dari satu state ke state lain. State akan disimpan sebagai sebuah list, dimana setiap kendaraan mendapat satu elemen (dictionary) berupa pasangan nama dan nilainya dengan isi sebagai berikut:

Tabel 2.1.1.1 Representasi State

Isi	Arti
`vehicle_id`	Pengenal kendaraan, misalnya A, B, C
`on_board`	Bernilai benar jika kendaraan diangkut, salah jika tidak
`x, y`	Posisi petak pojok kiri atas kendaraan di geladak
`orientation`	Orientasi: `H` (horizontal) atau `V` (vertikal)

Dengan sistem koordinat memusatkan (0,0) di pojok kiri atas (paling atas geladak) dengan sumbu X bergerak ke kanan dan sumbu y bergerak ke bawah. Pada orientasi H, lebar kendaraan searah sumbu x dan panjangnya searah sumbu y. Orientasi V menukar keduanya. Dengan aturan ini, petak yang ditempati sebuah

kendaraan dapat dihitung dari x, y, ukurannya, dan orientasinya. Untuk mempermudah pengertian, dapat melihat contoh dibawah ini.

Condition, vehicle A diangkut, B tidak, dan C diangkut. Maka state nya akan menjadi seperti berikut:

Tabel 2.1.1.2 Contoh Representasi State

vehicle_id	on_board	x	y	orientation	Petak
A	benar	0	0	H	x 0–1, y 0–1
B	salah	–	–	–	tidak ada
C	benar	0	2	H	x 0–3, y 2

Kendaraan yang tidak diangkut tetap punya elemen di daftar. Dengan begitu, ukuran daftar selalu sama dengan jumlah kendaraan, dan kendaraan dapat diangkut atau diturunkan hanya dengan mengubah on_board.

2.1.2. Penanganan Batasan

Penanganan batasan digunakan dengan mengizinkan *state* yang melanggar dengan memberinya penalti, yaitu pengurang nilai *state* objek saat ini. Perpindahan antar-*state* berupa perubahan kecil, misalnya menggeser atau memutar satu kendaraan. Perubahan kecil seperti itu sering melewati *state* yang melanggar sebelum sampai ke *state* yang lebih baik. Jika *state* yang melanggar dilarang, pencarian bisa terjebak. Dengan penalti, *state* yang melanggar tetap boleh dikunjungi, tetapi nilainya turun sehingga algoritma cenderung menjauhinya. Penalti yang diberikan adalah sebagai berikut

Tabel 2.1.2.1 Contoh Representasi State

Batasan	Perhitungan	Penalti
Tidak bertumpuk	Jumlah petak yang ditempati lebih dari satu kendaraan	200 per petak
Di dalam geladak	Jumlah petak yang berada di	200 per petak

	luar geladak	
Kapasitas berat	Selisih total berat dan kapasitas, jika berlebih	200 per satuan berat

Penalti dihitung hanya untuk kendaraan yang on_board. Besarnya penalti sebanding dengan banyaknya pelanggaran, sehingga state yang hampir sah mendapat penalti lebih kecil daripada state yang melanggar parah. Itu memberi petunjuk arah kepada algoritma.

2.1.3. Fungsi Objective

Fungsi objective memberikan angka daripada sebuah state, dimana semakin besar angkanya menunjukkan semakin baik pula state nya. Fungsi objective utama menilai dari total *Shipping Fee* yang dikenakan dikurangi dengan penalti (pada sub-bab sebelum ini), dengan rumus sebagai berikut:

$$\text{nilai}(state) = \Sigma\, fee_i \text{ (untuk kendaraan yang } \texttt{on_board}) - \text{penalti}(state)$$

Dengan nilai penalti memungkinkan 0 apabila tidak terjadi pelanggaran sama sekali.

Fungsi objective alternatif berbasis ETA (bonus). ETA (Estimated Time of Arrival) adalah perkiraan waktu tiba sebuah kendaraan. Pada fungsi alternatif ini, kendaraan dengan ETA lebih kecil dianggap lebih mendesak, sehingga *ShippingFee*-nya diberi bobot lebih besar dengan rumus:

$$\text{bobot}_i = 1 + (\text{ETAmaks} - \text{ETA}_i) / \text{ETAmaks}$$

Sehingga nilai ETA menjadi 1 <= ETA <= 2, dimana 1 sebagai nilai ETA tebesar dan 2 sebagai nilai ETA terkecil. Nilai state menjadi:

$$\text{nilai_ETA}(state) = \Sigma\, (fee_i \times \text{bobot}_i) \text{ (untuk kendaraan yang } \texttt{on_board}) - \text{penalti}(state)$$

Dengan cara perhitungan penalti yang sama persis terhadap fungsi utamanya

2.1.4. State Awal dan Langkah Neighbor

Pencarian harus dimulai dari sebuah state yang ditentukan secara acak: untuk setiap kendaraan, ditentukan secara acak apakah diangkut, posisinya di geladak, dan orientasinya.

Pengacakan memakai seed, yaitu angka awal untuk pembangkit bilangan acak. Dengan seed yang sama, urutan angka acaknya sama, sehingga state awal dan

seluruh jalannya pencarian bisa diulang persis. Ini penting agar hasil percobaan dapat direproduksi, dan agar perbandingan antar algoritma bisa dimulai dari state awal yang sama.

Neighbor (tetangga) adalah state lain yang didapat dengan satu perubahan kecil pada state sekarang. Algoritma pencarian lokal bergerak dari state ke salah satu neighbor-nya. Ada empat jenis perubahan:

Tabel 2.1.4.1 Contoh Representasi State

Isi	Arti
Swap	Dua kendaraan bertukar posisi
Relocate	Satu kendaraan dipindahkan ke posisi lain
Rotate	Orientasi satu kendaraan diputar antara `H` dan `V`
Toggle	Satu kendaraan dinaikkan ke kapal atau diturunkan (`on_board` dibalik)

Keempatnya cukup untuk menjangkau susunan apapun dimana toggle menentukan kendaraan yang diangkut, relocate dan swap menata letaknya, dan rotate mengatur arahnya. Karena langkah yang menghasilkan state melanggar batasan tetap diizinkan, setiap langkah selalu menghasilkan neighbor yang bisa dinilai.

Jumlah neighbor sangat besar, karena posisi relokasi punya banyak kemungkinan. Karena itu, neighbor tidak selalu dibuat semuanya. Hill-climbing menilai sejumlah neighbor acak pada setiap iterasi, sedangkan Simulated Annealing hanya mengambil satu neighbor acak. Algoritma Genetika tidak berpindah antar-neighbor, melainkan menggabungkan dua state dan memutasi satu kendaraan.

2.1.5. Data Uji dan Pengaturan Eksperimen

Agar berada dalam *environment* terkontrol, seluruh percobaan dilakukan menggunakan satu data uji `demo.json` yang dapat ditemukan pada *repository* github kelompok, dengan isi:

a. Geladak berukuran 12 × 10 petak, dengan kapasitas berat maksimum 120.
b. 12 kendaraan (A sampai L), masing-masing dengan ukuran, berat, *ShippingFee*, dan ETA. ETA hanya dipakai pada percobaan bonus.

Perhitungan maksimal dilakukan dengan batas atas, yaitu nilai yang tidak mungkin dilampaui oleh solusi sah manapun. Batas atas dihitung dengan mengabaikan bentuk dan letak kendaraan, hanya memilih kendaraan dengan total *ShippingFee* terbesar yang total beratnya tidak melebihi kapasitas (masalah *knapsack*). Karena bentuk dan tumpukan diabaikan, batas atas bisa lebih tinggi daripada optimum sebenarnya. Jadi selisih terhadap batas atas menunjukkan paling jauh seberapa dekat hasil kami dengan optimum.

Setiap algoritma akan dijalankan 3 kali dengan seed berbeda. Seed yang sama dipakai untuk semua algoritma, sehingga perbandingannya adil. Untuk Algoritma Genetika, ditambah dua variasi sesuai spek, masing-masing dijalankan 3 kali:

Tabel 2.1.5.1 Contoh Representasi State

Percobaan	**Yang divariasikan**	**Nilai**
Hill-climbing	–	[ISI: parameter, mis. `max_sideways`, `max_iter`]
Simulated Annealing	–	[ISI: suhu awal, laju pendinginan, iterasi]
GA, variasi iterasi	Jumlah generasi	[ISI: 3 nilai]
GA, variasi populasi	Ukuran populasi	[ISI: 3 nilai]

Untuk setiap percobaan dicatat: nilai akhir, *state* akhir, durasi pencarian, dan riwayat nilai per iterasi (untuk grafik). Semua hasil disimpan di folder `output/`.

2.1.6. Visualisasi

Visualisasi dibuat otomatis oleh program. Setelah sebuah algoritma selesai berjalan, hasilnya (initial_state, final_state, dan history) diteruskan ke modul

visualisasi (plot.py dan visualize.py) yang ditulis dengan Python. Modul ini menggambar dan menyimpan gambarnya sebagai berkas di folder output/, sehingga semua gambar di laporan berasal langsung dari hasil percobaan, bukan digambar manual. Ada dua jenis gambar:

a. Tampilan geladak. Geladak digambar sebagai grid. Setiap kendaraan yang diangkut digambar sebagai persegi panjang berwarna sesuai posisi, ukuran, dan orientasinya, lengkap dengan namanya. Gambar ini dibuat untuk state awal dan state akhir, sehingga perubahan akibat pencarian terlihat langsung. Kendaraan yang tidak diangkut didaftar terpisah di samping grid.
b. Grafik nilai per iterasi. Sumbu x adalah iterasi dan sumbu y adalah nilai state pada iterasi itu, diambil dari history. Grafik ini menunjukkan kecepatan naiknya nilai, kapan pencarian berhenti membaik, dan bentuk khas tiap algoritma. Pada Simulated Annealing nilai bisa sempat turun, sedangkan pada Hill-climbing nilai tidak pernah turun.

2.2. Hill Climbing

Hill-climbing adalah algoritma pencarian lokal yang paling sederhana. Algoritma ini mulai dari satu solusi, lalu terus berpindah ke solusi tetangga yang lebih baik sampai tidak ada lagi yang lebih baik. Algoritma ini ringan dan cepat, tetapi bisa berhenti di solusi yang bukan terbaik secara keseluruhan.

2.2.1. Cara Kerja Umum

Algoritma memulai dari satu *state* lalu berulang kali pindah ke *neighbor* yang nilainya lebih baik, seperti pendaki yang selalu melangkah ke arah yang lebih tinggi. Algoritma ini tidak menyimpan *state* lama dan tidak pernah turun, sehingga ringan dan cepat. Kelemahannya, pendaki bisa berhenti di puncak kecil yang bukan puncak tertinggi. Puncak kecil ini disebut *local optimum*, yaitu *state* yang nilainya lebih tinggi daripada semua *neighbor*-nya, tetapi belum tentu yang terbaik secara keseluruhan.

Variasi yang kami pakai adalah *sideways move*, yaitu izin untuk pindah ke *neighbor* yang nilainya sama dengan *state* sekarang. Di persoalan ini, banyak perubahan yang tidak mengubah nilai, misalnya menggeser kendaraan ke tempat

lain yang sama-sama sah. Daerah datar seperti itu bisa menjebak algoritma biasa, padahal di ujung daerah datar mungkin ada jalan naik. Dengan *sideways move*, algoritma boleh berjalan menyusuri daerah datar untuk mencari jalan naik.

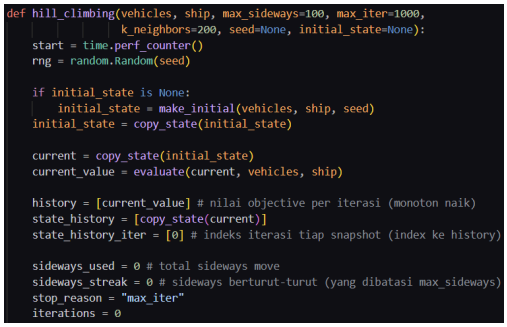
Berikut merupakan langkah kerja pada setiap iterasi yang terjadi:

a. Dari state sekarang, buat sejumlah neighbor dan hitung nilai masing-masing.
b. Ambil neighbor bernilai tertinggi. Jika beberapa nilainya sama-sama tertinggi, pilih salah satu secara acak.
c. Bandingkan dengan nilai state saat ini
    i. Lebih tinggi: pindah ke neighbor itu, dan hitungan sideways move beruntun diulang dari nol.
    ii. Sama: pindah juga (sideways move), dan hitungan sideways move beruntun bertambah satu.
    iii. Lebih rendah: berhenti, karena tidak ada *neighbor* yang lebih baik.
d. Ulangi dari langkah pertama

2.2.2. Source Code

Hill-climbing diimplementasikan di fungsi `hill_climbing` pada berkas `src/hill_climbing.py`. Fungsi ini memakai `random_neighbors` dari `src/neighbor_hcsa.py` untuk membuat *neighbor*, dan `evaluate` dari `src/objective.py` untuk menilai *state*.

Tabel 2.2.2.1 Source Code Hill Climbing

Code	Fungsi
	Fungsi menerima data kendaraan dan kapal, lalu empat parameter yang bisa diatur: `max_sideways` (batas *sideways move* beruntun, bawaan 100), `max_iter` (batas iterasi, bawaan 1000), `k_neighbors` (jumlah *neighbor* yang dibuat per iterasi, bawaan 200), dan `seed` (angka awal pembangkit

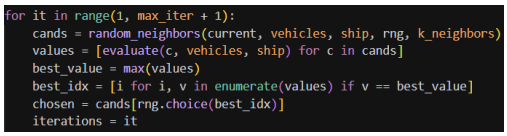
```
def hill_climbing(vehicles, ship, max_sideways=100, max_iter=1000,
                  k_neighbors=200, seed=None, initial_state=None):
    start = time.perf_counter()
    rng = random.Random(seed)

    if initial_state is None:
        initial_state = make_initial(vehicles, ship, seed)
    initial_state = copy_state(initial_state)

    current = copy_state(initial_state)
    current_value = evaluate(current, vehicles, ship)

    history = [current_value] # nilai objective per iterasi (monoton naik)
    state_history = [copy_state(current)]
    state_history_iter = [0] # indeks iterasi tiap snapshot (index ke history)

    sideways_used = 0 # total sideways move
    sideways_streak = 0 # sideways berturut-turut (yang dibatasi max_sideways)
    stop_reason = "max_iter"
    iterations = 0
```

	bilangan acak agar hasil bisa diulang). Parameter `initial_state` bersifat opsional. Jika tidak diisi, *state* awal dibuat acak dari `seed`: setiap kendaraan diangkut dengan peluang 70%, diberi posisi acak di geladak, dan orientasi horizontal atau vertikal dengan peluang sama. `history` mencatat nilai di setiap iterasi untuk membuat grafik, dan `state_history` menyimpan *state* setiap kali nilainya membaik. `sideways_used` menghitung semua *sideways move*, sedangkan `sideways_streak` hanya menghitung yang beruntun.
	Pada setiap iterasi dibuat `k_neighbors` *neighbor* acak, lalu nilai masing-masing dihitung. Nilai tertinggi diambil. Jika beberapa *neighbor* sama-sama tertinggi, salah satunya dipilih secara acak lewat `rng.choice`. Perlu dicatat bahwa yang dinilai hanya sampel acak, bukan semua *neighbor* yang mungkin.

``
if best_value > current_value:
    current, current_value = chosen, best_value
    sideways_streak = 0
    state_history.append(copy_state(current))
    state_history_iter.append(it)
elif best_value == current_value:
    current = chosen
    sideways_used += 1
    sideways_streak += 1
    if sideways_streak > max_sideways:
        history.append(current_value)
        stop_reason = "max_sideways"
        break
else:
    history.append(current_value)
    stop_reason = "local_optimum"
    break
history.append(current_value)
``
	Nilai *neighbor* terbaik dibandingkan dengan nilai *state* sekarang. Jika lebih tinggi, algoritma pindah ke *neighbor* itu, dan hitungan *sideways move* beruntun diulang dari nol. Jika sama, algoritma tetap pindah (*sideways move*), hitungan bertambah satu, dan pencarian berhenti dengan alasan `max_sideways` bila hitungan beruntun melebihi batas. Jika lebih rendah, pencarian berhenti dengan alasan `local_optimum`, karena tidak ada *neighbor* yang lebih baik. Jika tidak ada kondisi yang menghentikan, pencarian berakhir sendiri setelah `max_iter` iterasi.

``
duration = time.perf_counter() - start
return {
    "initial_state": initial_state,
    "final_state": current,
    "history": history,
    "duration": duration,
    "metrics": {
        "algorithm": "hill_climbing_sideways",
        "final_value": current_value,
        "iterations": iterations,
        "sideways_used": sideways_used,
        "max_sideways": max_sideways,
        "k_neighbors": k_neighbors,
        "max_iter": max_iter,
        "stop_reason": stop_reason,
        "seed": seed,
    },
    "state_history": state_history,
    "state_history_iter": state_history_iter,
}
``
	Hasilnya berupa kamus berisi *state* awal dan akhir, riwayat nilai, durasi, dan ringkasan (`metrics`) yang memuat nilai akhir, jumlah iterasi, jumlah *sideways move*, parameter yang dipakai, dan alasan berhenti. Bentuk yang sama dipakai oleh semua algoritma, sehingga modul yang membuat grafik dan gambar bisa memakainya tanpa dibedakan.

```python
def random_neighbor(state, vehicles, ship, rng):
    """Satu neighbor acak dari state. Mengembalikan state baru (deep copy)."""
    n = len(state)
    on_idx = _on_board_indices(state)

    # kalau belum ada kendaraan di kapal, satu-satunya move bermakna: toggle masuk
    if on_idx:
        jenis = rng.choice(["move", "move", "rotate", "toggle", "swap"])
    else:
        jenis = "toggle"

    new_state = copy_state(state)

    if jenis == "move":
        i = rng.choice(on_idx)
        pos = _random_fit_position(new_state[i], vehicles, ship, rng)
        if pos is not None:
            new_state[i]["x"], new_state[i]["y"] = pos
        return new_state
```

```python
def random_neighbors(state, vehicles, ship, rng, k):
    """Sampel k neighbor acak (dipakai HC sebagai pengganti seluruh neighbor)."""
    return [random_neighbor(state, vehicles, ship, rng) for _ in range(k)]
```

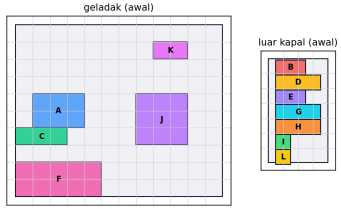
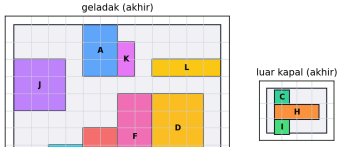
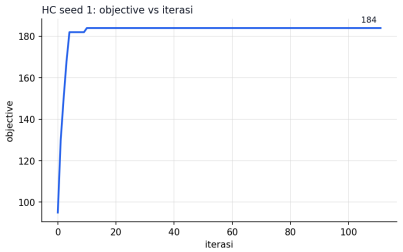
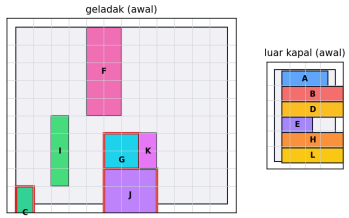
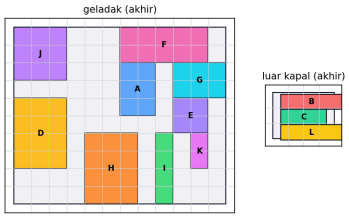
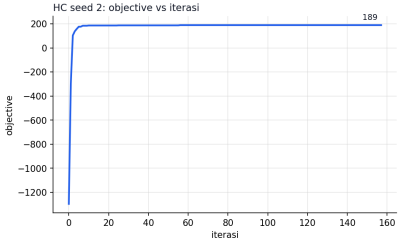
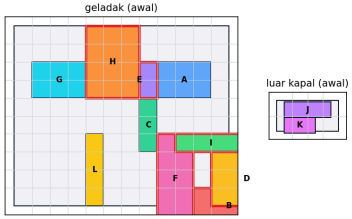
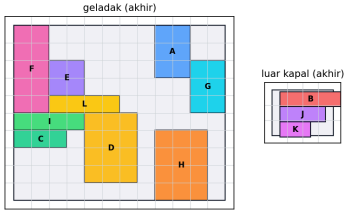
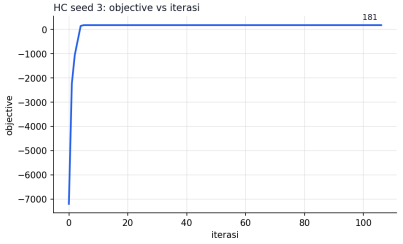
Satu neighbor dibuat dengan memilih satu jenis perubahan secara acak pada salinan state:

- move: memindahkan satu kendaraan yang sedang diangkut ke posisi acak yang pasti muat di dalam geladak.
- rotate: memutar orientasi satu kendaraan antara horizontal dan vertikal, lalu menggesernya bila setelah diputar ia keluar dari geladak.
- toggle: menaikkan kendaraan ke kapal atau menurunkannya. Kendaraan yang dinaikkan diberi posisi acak yang muat.
- swap: menukar seluruh isi dua kendaraan, yaitu status diangkut atau tidak, posisi, dan orientasi.

Jenis move ditulis dua kali di daftar pilihan, sehingga peluangnya 2 dari 5, dua kali lipat jenis lain. Jika belum ada kendaraan di kapal, satu-satunya langkah yang berarti adalah toggle. Karena perubahan dilakukan pada salinan, state sekarang tidak rusak. random_neighbors hanya mengulang proses ini sebanyak k kali.

2.2.3. Hasil Eksperimen Hill Climbing - Standar

Tabel 2.2.3.1 Hasil Eksperimen Hill Climbing

Seed	State Awal	State Akhir	Durasi	Nilai Awal	Nilai Akhir	Iterasi	Alasan berhenti	Grafik
1			3,710	95	184	111	max_sideways	
2			4,695	-1297	189	157	max_sideways	
3			2,810	-7199	181	106	max_sideways	

Rincian state akhir dari percobaan adalah sebagai berikut

Tabel 2.2.3.2 Hasil Eksperimen Hill Climbing (2)

Seed	Kendaraan Terangkut	Total Shipping Fee	Total Berat (maksimal 120)	Kendaraan diluar kapal	Validity
1	9	184	120	C, H, I	Ya
2	9	189	120	B, C, L	Ya
3	9	181	118	B, J, K	Ya

Ringkasan tiga percobaan: nilai akhir rata-rata 184,67 (terendah 181, tertinggi 189, selisih 8), dan durasi rata-rata 3,74 detik.

2.2.4. Varian Hill Climbing (Implementasi Bonus)

Selain Hill-climbing dengan *sideways move*, diimplementasikan tiga varian lain di berkas `src/hill_climbing_variants.py`: *steepest ascent*, *stochastic*, dan *random restart*. Seperti Hill-climbing sebelumnya, ketiganya membuat `k_neighbors` (bawaan 200) *neighbor* acak pada setiap iterasi lewat `random_neighbors`, lalu menilainya dengan `evaluate`. Perbedaannya hanya pada cara memilih *neighbor* dan kapan pencarian berhenti. Terdapat kode yang digunakan bersama oleh ketiga varians secara bersamaan

Tabel 2.2.4.1 Source Code Varians Hill Climbing

Code	Fungsi

`def _mendaki(pilih, vehicles, ship, rng, awal, max_iter, k):`
`  # naik terus selama masih ada tetangga yang lebih baik`
`  current = copy_state(awal)`
`  nilai_now = evaluate(current, vehicles, ship)`
`  values = [nilai_now]`
`  states = [copy_state(current)]`
`  alasan = "max_iter"`
 	
`  for _ in range(max_iter):`
`  nxt, nilai = pilih(current, nilai_now, vehicles, ship, rng, k)`
`  if nxt is None or nilai <= nilai_now:`
`  alasan = "local_optimum"`
`  values.append(nilai_now)`
`  states.append(copy_state(current))`
`  break`
`  current, nilai_now = nxt, nilai`
`  values.append(nilai_now)`
`  states.append(copy_state(current))`
 	
`  return current, nilai_now, values, states, alasan`
	Fungsi ini menerima cara memilih *neighbor* sebagai parameter `pilih`, sehingga logika pendakiannya ditulis sekali dan dipakai ketiga varian. Pada setiap iterasi, *neighbor* yang dipilih diperiksa. Jika tidak ada (`None`) atau nilainya tidak lebih tinggi, pencarian berhenti dengan alasan `local_optimum`. Jika lebih tinggi, algoritma pindah dan mencatat nilainya. Jika tidak pernah berhenti, pencarian berakhir setelah `max_iter` (1000) iterasi.

a. Steepest Ascent

Steepest ascent selalu memilih neighbor bernilai tertinggi, lalu pindah hanya jika nilainya lebih tinggi dari state sekarang. Berbeda dengan Hill-climbing sideways move, varian ini tidak pernah berpindah ke

neighbor bernilai sama, sehingga pencarian berhenti begitu tidak ada neighbor yang lebih baik.

Tabel 2.2.4.1 Source Code Steepest Ascent

Code	Fungsi

`def _pilih_steepest(current, nilai_now, vehicles, ship, rng, k):`
`  # ambil tetangga terbaik, kalau seri pilih acak`
`  kandidat = random_neighbors(current, vehicles, ship, rng, k)`
`  nilai = [evaluate(c, vehicles, ship) for c in kandidat]`
`  terbaik = max(nilai)`
`  seri = [i for i, v in enumerate(nilai) if v == terbaik]`
`  return kandidat[rng.choice(seri)], terbaik`
	Fungsi ini membuat 200 neighbor, menilai semuanya, dan mengembalikan yang tertinggi beserta nilainya. Jika beberapa neighbor sama-sama tertinggi, salah satunya dipilih secara acak.

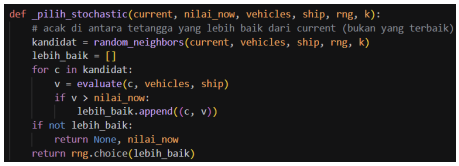
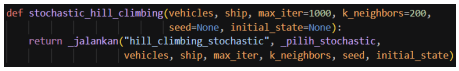
`def steepest_ascent(vehicles, ship, max_iter=1000, k_neighbors=200, seed=None,`
`  initial_state=None):`
`  return _jalankan("hill_climbing_steepest_ascent", _pilih_steepest,`
`  vehicles, ship, max_iter, k_neighbors, seed, initial_state)`
	Fungsi `_jalankan` membuat *state* awal acak dari `seed`, menjalankan `_mendaki` dengan cara pemilihan yang diberikan, lalu mengemas hasilnya dalam bentuk yang sama dengan algoritma lain (*state* awal dan akhir, riwayat nilai, durasi, ringkasan).

b. Stochastic Hill Climbing

Stochastic hill-climbing tidak mengambil neighbor terbaik. Ia mengumpulkan semua neighbor yang lebih baik dari state sekarang, lalu memilih satu di antaranya secara acak. Jika tidak ada yang lebih baik, pencarian berhenti. Akibatnya, langkahnya lebih bervariasi dan tidak selalu mengambil kenaikan terbesar.

Tabel 2.2.4.2 Source Code Stochastic Hill Climbing

Code	Fungsi

	Setiap *neighbor* dinilai dan hanya yang nilainya lebih tinggi disimpan. Jika daftarnya kosong, fungsi mengembalikan `None` sebagai tanda berhenti. Jika tidak, satu pasangan *neighbor* dan nilainya diambil secara acak.
	Pendakian dan pembungkusnya memakai `_mendaki` dan `_jalankan` yang sama, hanya dengan `_pilih_stochastic` sebagai cara memilih (fungsi `stochastic_hill_climbing`)

c. Random Restart Hill Climbing

Random restart mengatasi kelemahan Hill-climbing yang mudah berhenti di local optimum dengan mengulang pendakian dari awal. Setiap pengulangan (restart) adalah satu kali steepest ascent dari state awal acak yang baru. Hasil akhirnya adalah state terbaik dari seluruh restart.

Tabel 2.2.4.3 Source Code Random Restart Hill Climbing

Code	Fungsi

```python
def random_restart_hill_climbing(vehicles, ship, max_restart=10, max_iter=1000,
                                 k_neighbors=200, seed=None, initial_state=None):
    # tiap restart = 1 run steepest ascent dari state awal acak baru,
    # hasil akhirnya yang terbaik dari semua restart
    t0 = time.perf_counter()
    rng = random.Random(seed)
    base_seed = 0 if seed is None else seed

    awal_pertama = _state_awal(vehicles, ship, seed, initial_state)
    best = None
    best_value = None
    history = []
    state_history = []
    state_iter = []
    iter_per_restart = []

    for r in range(max_restart):
        if r == 0:
            awal = awal_pertama
        else:
            awal = make_initial(vehicles, ship, base_seed * 1000 + r)

        _, _, values, states, _ = _mendaki(
            _pilih_steepest, vehicles, ship, rng, awal, max_iter, k_neighbors)
        iter_per_restart.append(len(values) - 1)

        # gabung ke history global, isinya nilai terbaik sejauh ini
        for i, v in enumerate(values):
            if r > 0 and i == 0:
                continue  # nilai awal restart baru bukan iterasi
            if best_value is None or v > best_value:
                best_value = v
                best = copy_state(states[i])
                history.append(best_value)
                state_history.append(copy_state(best))
                state_iter.append(len(history) - 1)
            else:
                history.append(best_value)

    metrics = {
        "algorithm": "hill_climbing_random_restart",
        "final_value": best_value,
        "iterations": len(history) - 1,
        "restarts": max_restart,
        "max_restart": max_restart,
        "iterations_per_restart": iter_per_restart,
        "k_neighbors": k_neighbors,
        "max_iter": max_iter,
        "stop_reason": "max_restart",
        "seed": seed,
    }
```

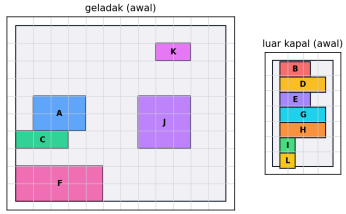
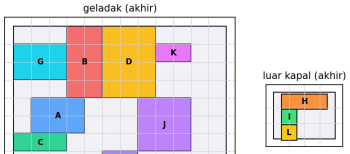
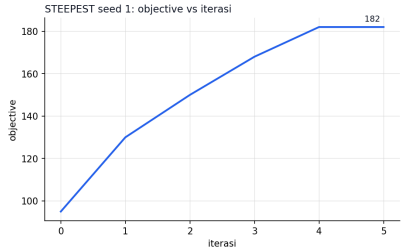
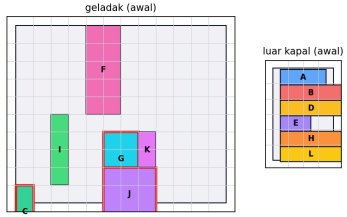
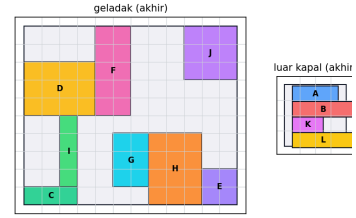
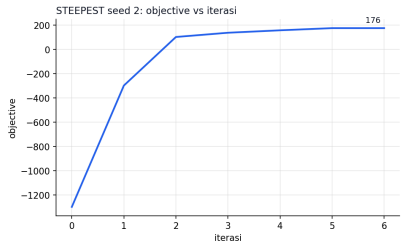
Parameter tambahannya adalah `max_restart` (jumlah pengulangan, bawaan 10). *Restart* pertama dimulai dari *state* awal yang dibuat dari `seed`, sedangkan *restart* berikutnya dimulai dari *state* acak baru yang dibuat dari angka `seed * 1000 + r`, sehingga hasilnya tetap bisa diulang. Pada setiap *restart* dijalankan `_mendaki` dengan `_pilih_steepest`. Bagian pencatatan (ditandai `...`) menyimpan nilai dan *state* terbaik dari semua *restart*, dan riwayat nilainya berupa "terbaik sejauh ini", sehingga grafiknya tidak pernah turun. Pencarian berhenti setelah `max_restart` pengulangan.

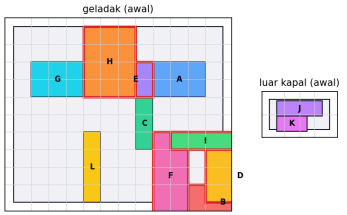
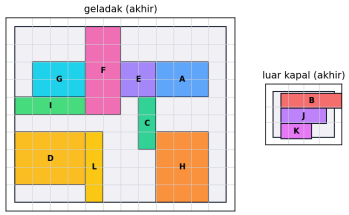
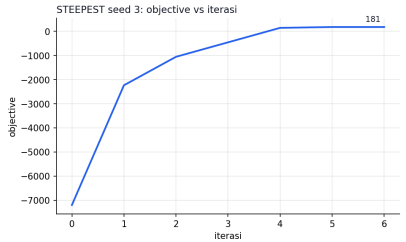
2.2.5. Hasil Eksperimen Varians Hill Climbing

Ketiga varian dijalankan pada data dan *seed* yang sama dengan Hill-climbing dasar (`demo.json`; *seed* 1, 2, 3), sehingga *state* awal tiap *seed* sama. Parameter: `max_iter` = 1000 dan `k_neighbors` = 200 untuk ketiganya, serta `max_restart` = 10 untuk *random restart*. Batas atas nilai pada data ini adalah 189.

2.2.5.1. Steepest Ascent

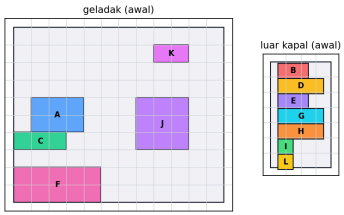
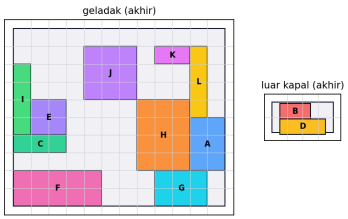
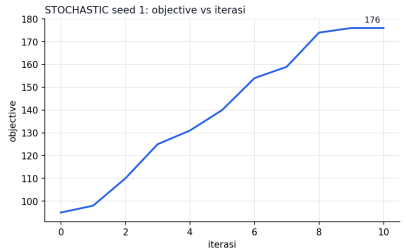
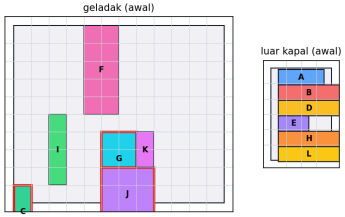
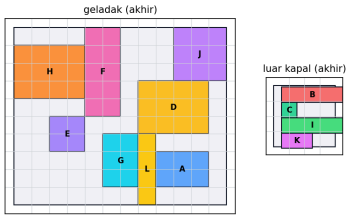
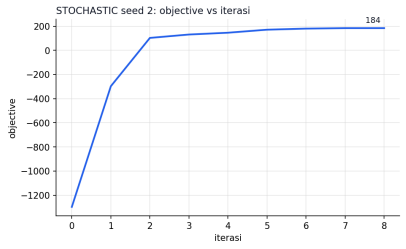
Tabel 2.2.5.1.1 Hasil Eksperimen Hill Climbing

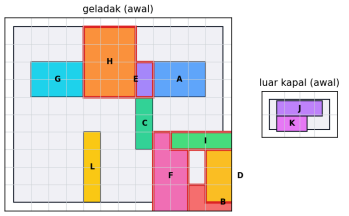
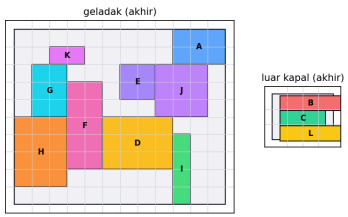
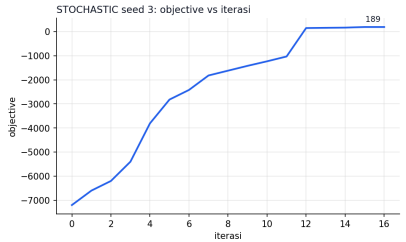
Seed	State Awal	State Akhir	Durasi	Nilai Awal	Nilai Akhir	Iterasi	Alasan berhenti	Grafik
1			0,074	95	182	5	local_optimum	
2			0,093	-1297	176	6	local_optimum	

3			0,108	-7199	181	6	local_optimum	

2.2.5.2. Stochastic Hill Climbing

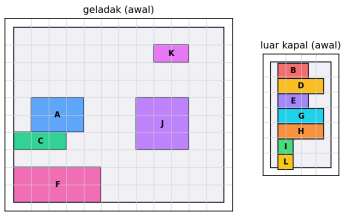
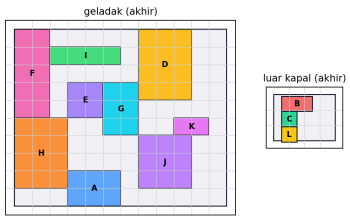
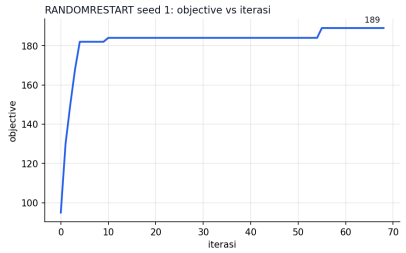
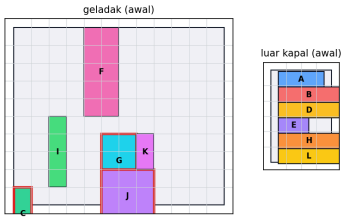
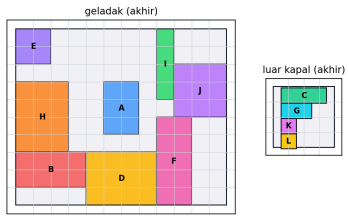
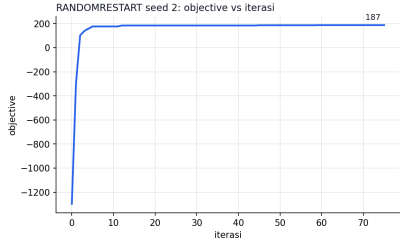
Tabel 2.2.5.2.1 Hasil Eksperimen Hill Climbing

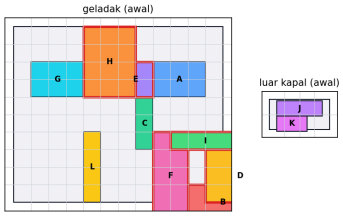
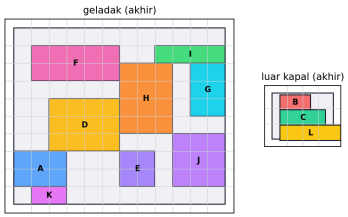
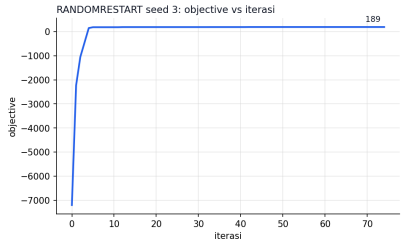
Seed	State Awal	State Akhir	Durasi	Nilai Awal	Nilai Akhir	Iterasi	Alasan berhenti	Grafik
1			0,154	95	176	10	local_optimum	
2			0,122	-1297	184	8	local_optimum	

3			0,352	-7199	189	16	local_optimum	

2.2.5.3. Random Restart Hill Climbing

Tabel 2.2.5.3.1 Hasil Eksperimen Hill Climbing

Seed	State Awal	State Akhir	Durasi	Nilai Awal	Nilai Akhir	Iterasi	Alasan berhenti	Grafik
1			1,216	95	189	68	max_restart	
2			1,225	-1297	187	75	max_restart	

3			1,253	-7199	189	74	max_restart	

Dengan detail iterasi sebagai berikut

Tabel 2.2.5.3.2 Banyak Iterasi

Seed	Iterasi per Restart (ke-1 hingga ke -10)
1	5, 6, 8, 9, 8, 4, 9, 7, 6, 6
2	6, 7, 8, 8, 9, 8, 7, 7, 8, 7
3	6, 7, 8, 7, 6, 9, 7, 7, 8, 9

Untuk merangkum seluruh hasil eksperimen (varians), dapat dilihat tabel berikut

Tabel 2.2.5.1 Hasil Eksperimen Hill Climbing

Varian	***Seed***	**Nilai awal**	**Nilai akhir**	**Selisih dari 189**	**Iterasi**	**Durasi (detik)**	**Alasan berhenti**	**Kendaraan diangkut**	**Total berat (maks 120)**
HC *sideways move*	1	95	184	5	111	3,710	`max_sideways`	9	120
	2	−1297	189	0	157	4,695	`max_sideways`	9	120

	3	−7199	181	8	106	2,810	max_sideways	9	118
	Rata-rata		**184,67**	**4,33**	**124,67**	**3,74**			
Steepest ascent	1	95	182	7	5	0,074	local_optimum	9	119
	2	−1297	176	13	6	0,093	local_optimum	8	113
	3	−7199	181	8	6	0,108	local_optimum	9	118
	Rata-rata		**179,67**	**9,33**	**5,67**	**0,09**			
Stochastic	1	95	176	13	10	0,154	local_optimum	10	117
	2	−1297	184	5	8	0,122	local_optimum	8	118
	3	−7199	189	0	16	0,352	local_optimum	9	120
	Rata-rata		**183,00**	**6,00**	**11,33**	**0,21**			
Random restart (10 *restart*)	1	95	189	0	68	1,216	max_restart	9	120
	2	−1297	187	2	75	1,225	max_restart	8	120
	3	−7199	189	0	74	1,253	max_restart	9	120

	Rata-rata		**188,33**	**0,67**	**72,33**	**1,23**			

2.2.6. Pembahasan Hasil

Setelah mencoba 4 cara dalam melakukan hill climbing, ditemukan data (perbandingan) sebagai berikut:

Tabel 2.2.6.1 Hasil Eksperimen Hill Climbing

	HC *sideways move*	***Steepest ascent***	***Stochastic***	***Random restart***
Cara memilih *neighbor*	Terbaik dari sampel; boleh pindah ke yang bernilai sama	Terbaik dari sampel; hanya pindah jika lebih baik	Acak di antara yang lebih baik	Terbaik dari sampel, diulang dari 10 *state* awal
Nilai akhir (*seed* 1, 2, 3)	184, 189, 181	182, 176, 181	176, 184, 189	189, 187, 189
Rata-rata nilai akhir	184,67	179,67	183,00	188,33
Rata-rata dari optimum	97,7%	95,1%	96,8%	99,6%
Terendah – tertinggi	181 – 189	176 – 182	176 – 189	187 – 189
Selisih terendah–tertinggi	8	6	13	2
Percobaan yang mencapai 189	1 dari 3	0 dari 3	1 dari 3	2 dari 3
Rata-rata iterasi	124,67	5,67	11,33	72,33
Rata-rata durasi (detik)	3,74	0,09	0,21	1,23
Alasan berhenti	`max_sideways`	`local_optimum`	`local_optimum`	`max_restart`

Secara keseluruhan, *random restart* memberi hasil terbaik: rata-rata paling dekat dengan optimum (99,6%), paling konsisten (selisih hanya 2), dan paling sering mencapai optimum (2 dari 3 percobaan), dengan waktu sekitar 1,2 detik. *Steepest*

ascent paling cepat (0,09 detik) tetapi paling jauh dari optimum, sedangkan HC dasar dan *stochastic* berada di tengah dengan kelebihan dan kekurangan yang berbeda. HC dasar paling lama (3,74 detik) karena banyak iterasi di dataran datar, dan *stochastic* paling tidak konsisten (selisih 13). Seluruh 12 percobaan menghasilkan susunan yang sah, dan nilai terendah (176) masih sekitar 7% di bawah optimum. Ada pertukaran yang jelas antara kualitas dan waktu: varian yang menambah usaha (iterasi di dataran datar atau pengulangan dari titik awal baru) menghasilkan nilai yang lebih baik.

HC dasar dan *steepest ascent* mulai dari *state* yang sama dan menempuh langkah yang sama persis sampai *steepest ascent* berhenti. Bedanya, *steepest ascent* berhenti saat tidak ada *neighbor* yang lebih baik, sedangkan HC dasar masih boleh berjalan ke *neighbor* bernilai sama. Dari titik berhenti itu, HC dasar menemukan jalan naik pada dua *seed* (182 menjadi 184, dan 176 menjadi 189) dan tidak menemukannya pada satu *seed* (181 tetap 181). Karena langkahnya sama sampai titik itu, selisih hasil pada dua *seed* tersebut dapat dikaitkan langsung dengan *sideways move*. Harganya adalah waktu. HC dasar menjalankan 106 sampai 157 iterasi, dengan 101 sampai 148 di antaranya berupa *sideways move*, dan nilai terbaiknya sudah dicapai jauh sebelum pencarian berhenti (iterasi ke-10, ke-56, dan ke-5). Seluruh percobaan berhenti karena batas 100 *sideways move* beruntun terlampaui. Satu *seed* mencapai optimum (189), dua lainnya kurang 5 dan 8.

Di lain sisi, varian steepest ascent merupakan yang paling sederhana: selalu memilih *neighbor* terbaik dan berhenti begitu tidak ada yang lebih baik. Hanya 5 sampai 6 iterasi dan sekitar 0,09 detik yang dibutuhkan, tetapi hasilnya paling rendah (rata-rata 179,67, tidak ada yang mencapai optimum). Tanpa kemampuan untuk berjalan di dataran datar, pencarian mudah terhenti di titik yang masih jauh dari optimum, seperti terlihat pada *seed* 2 yang berhenti di 176 sementara HC dasar dari *state* yang sama berhasil naik sampai 189.

Berbeda dengan *stochastic hill climbing* yang tidak mengambil kenaikan terbesar, melainkan memilih satu *neighbor* yang lebih baik secara acak. Akibatnya iterasinya lebih banyak (8 sampai 16) daripada *steepest ascent*, dan rata-ratanya lebih tinggi (183,00 dibanding 179,67). Namun sebarannya paling lebar: satu percobaan mencapai optimum (189) dan satu lagi hanya 176. Dengan hanya tiga percobaan, belum bisa disimpulkan bahwa *stochastic* lebih baik daripada *steepest ascent*. Yang terlihat jelas hanyalah bahwa hasilnya lebih bervariasi dan tetap berhenti di *local optimum*.

Di lain sisi, *random restart* memanfaatkan kenyataan bahwa hasil satu kali *steepest ascent* sangat bergantung pada titik awalnya. Pada 10 *restart* tiap *seed*, nilai akhirnya bervariasi, misalnya 174 sampai 189 pada *seed* 1. Dengan mengambil yang terbaik dari 10 percobaan itu, hasilnya lebih tinggi dan lebih stabil. Nilai terbaik juga tidak selalu datang dari *restart* pertama (*restart* ke-8, ke-8, dan ke-6), sehingga *restart* tambahan memang memberi peluang tambahan. Biayanya sebanding dengan jumlah *restart*: durasinya sekitar 13 kali *steepest ascent*, yang sejalan dengan jumlah iterasinya (72 dibanding 5,67). Meski begitu, waktunya masih lebih singkat daripada HC dasar pada percobaan ini.

2.3. Simulated Annealing

Simulated Annealing (SA) adalah algoritma pencarian lokal yang meniru proses annealing pada logam. Logam dipanaskan lalu didinginkan pelan-pelan supaya susunan atomnya sampai ke keadaan berenergi rendah. Hill Climbing berhenti ketika semua neighbor yang dicoba lebih buruk dari state sekarang. SA tetap boleh pindah ke state yang lebih buruk dengan peluang tertentu, sehingga pencarian bisa keluar dari local optimum.

2.3.1. Cara Kerja Umum

Satu iterasi SA berjalan melalui proses berikut.

a. Ambil satu neighbor acak dari state sekarang.

b. Hitung ΔE = nilai(neighbor) − nilai(state sekarang).
c. Jika $\Delta E \geq 0$, neighbor diterima.
d. Jika $\Delta E < 0$, neighbor diterima dengan peluang e^(ΔE/T). Peluang ini dibandingkan dengan angka acak antara 0 dan 1
e. Suhu diturunkan: $T \leftarrow \alpha \cdot T$.

Pencarian berhenti ketika T < T_min atau iterasi mencapai max_iter. *State* yang dikembalikan adalah state terbaik yang pernah dikunjungi, bukan state terakhir, karena state terakhir bisa saja lebih buruk dari yang pernah ditemukan.

Suhu mengatur seberapa mudah langkah buruk diterima. Misalkan sebuah langkah mengeluarkan kendaraan bernilai 20 sehingga $\Delta E = -20$. Pada T = 86,27 langkah itu diterima dengan peluang 0,79. Pada T = 10 peluangnya 0,14, dan pada T = 1 peluangnya sekitar 2×10^{-9}. Jadi di awal SA menjelajah dengan bebas, dan di akhir ia nyaris hanya menerima langkah yang tidak memperburuk nilai.

Parameter yang kami pakai:

- Suhu awal T0 dihitung dari data. Langkah buruk yang paling wajar pada persoalan ini adalah mengeluarkan satu kendaraan, yang menurunkan nilai sebesar fee kendaraan itu. T0 dipilih supaya langkah dengan penurunan sebesar rata-rata fee diterima dengan peluang p0 = 0,8 di awal: T0 = rata-rata fee / −ln(p0). Rata-rata fee 12 kendaraan pada demo.json adalah 19,25, sehingga T0 = 86,27.
- Pendinginan geometrik dengan faktor α. Nilai α dihitung supaya suhu turun dari T0 ke T_min = 0,1 tepat setelah max_iter = 20.000 iterasi: α = (T_min / T0)^(1/max_iter) = 0,999662.
- Stuck didefinisikan sebagai iterasi ketika kandidat yang lebih buruk ditolak ($\Delta E < 0$ dan angka acak ≥ e^(ΔE/T)). Frekuensi stuck adalah jumlah iterasi stuck dibagi jumlah iterasi.
- State awal dan neighbor dibuat dengan fungsi yang sama seperti pada Hill Climbing (bagian 2.1.4). Bedanya, SA hanya mengambil satu neighbor di tiap iterasi, tidak sekumpulan neighbor.

2.3.2. Source Code

Simulated Annealing diimplementasikan di fungsi simulated_annealing pada berkas src/simulated_annealing.py. Fungsi ini memakai calibrate_t0 di berkas yang sama untuk menentukan suhu awal, make_initial dan random_neighbor dari src/neighbor_hcsa.py untuk membuat state awal dan neighbor, dan evaluate dari src/objective.py untuk menilai state.

Tabel 2.3.2.1 Source Code Simulated Annealing

Code	Fungsi

`def calibrate_t0(vehicles, p0=0.8):`
`  # Menentukan suhu awal T0.`
`  # Langkah buruk yang paling wajar adalah mengeluarkan satu kendaraan dari`
`  # kapal, yang menurunkan nilai sebesar fee kendaraan itu.`
`  # Kita ingin langkah itu diterima dengan peluang p0 (0.8) di awal:`
`  #  e^(-fee_rata_rata / T0) = p0  ->  T0 = fee_rata_rata / -ln(p0)`
`  total_fee = 0`
`  for v in vehicles.values():`
`  total_fee = total_fee + v["shipping_fee"]`
`  fee_rata_rata = total_fee / len(vehicles)`
`  return fee_rata_rata / -math.log(p0)`
	Fungsi calibrate_t0 menentukan suhu awal. Fungsi ini menjumlahkan shipping_fee semua kendaraan, membaginya dengan jumlah kendaraan untuk mendapat fee rata-rata, lalu membaginya dengan −ln(p0). Dengan p0 = 0,8, langkah buruk sebesar fee rata-rata diterima dengan peluang 0,8 di awal pencarian. Pada demo.json rata-rata fee adalah 19,25, sehingga T0 = 86,27.

`def simulated_annealing(vehicles, ship, T0=None, alpha=None, T_min=0.1,`
`  max_iter=20000, seed=None, initial_state=None,`
`  p0=0.8):`
`  start = time.perf_counter()`
`  rng = random.Random(seed)`
 	
`  # State awal: kalau tidak diberikan, buat acak dari seed.`
`  if initial_state is None:`
`  initial_state = make_initial(vehicles, ship, seed)`
`  initial_state = copy_state(initial_state)`
 	
`  # Suhu awal dan laju pendinginan.`
`  # Kalau alpha tidak diisi, dihitung supaya suhu turun dari T0 sampai T_min`
`  # tepat setelah max_iter iterasi.`
`  if T0 is None:`
`  T0 = calibrate_t0(vehicles, p0)`
`  if alpha is None:`
`  if T0 > T_min:`
`  alpha = (T_min / T0) ** (1.0 / max_iter)`
`  else:`
`  alpha = 0.995`
	Fungsi menerima data kendaraan dan kapal, lalu parameter yang bisa diatur: T0 (suhu awal), alpha (faktor pendinginan), T_min (suhu minimum, bawaan 0,1), max_iter (batas iterasi, bawaan 20000), seed (angka awal pembangkit bilangan acak), initial_state (opsional), dan p0 (peluang awal untuk kalibrasi, bawaan 0,8).
	
	Pembangkit angka acak dibuat dari random.Random(seed), sehingga hasil bisa diulang dan random global tidak terganggu. Jika initial_state tidak diberikan, state awal dibuat acak dari seed

	oleh make_initial, lalu disalin supaya data asli tidak berubah. T0 dan alpha boleh dikosongkan. Jika T0 kosong, nilainya dihitung oleh calibrate_t0. Jika alpha kosong, nilainya dihitung supaya suhu turun dari T0 ke T_min tepat setelah max_iter iterasi: alpha = (T_min / T0)^(1/max_iter). Jika T0 tidak lebih besar dari T_min, alpha memakai nilai cadangan 0,995.

`current = copy_state(initial_state)`
`current_value = evaluate(current, vehicles, ship)`
`best = copy_state(current)`
`best_value = current_value`
 	
`# Catatan yang dikembalikan ke pemanggil`
`history = [best_value]  # nilai TERBAIK sejauh ini per iterasi`
`history_current = [current_value]  # nilai state SAAT INI per iterasi`
`accept_prob_history = []  # min(1, e^(dE/T)) per iterasi`
`temperature_history = [T0]`
`state_history = [copy_state(best)]`
`state_history_iter = [0]`
 	
`stuck_count = 0  # berapa kali langkah buruk ditolak`
`accepted_worse = 0  # berapa kali langkah buruk diterima`
`T = T0`
`iterations = 0`
	current adalah state yang sedang ditempati dan best adalah state terbaik yang pernah dikunjungi. history menyimpan nilai terbaik per iterasi, history_current menyimpan nilai state sekarang, accept_prob_history menyimpan min(1, e^(ΔE/T)) per iterasi untuk grafik peluang, dan temperature_history menyimpan suhu. state_history menyimpan state setiap kali nilai terbaik membaik. stuck_count menghitung langkah buruk yang ditolak dan accepted_worse menghitung langkah buruk yang diterima.

`for it in range(1, max_iter + 1):`
`if T < T_min:`
`  break  # sudah cukup dingin`
 	
`candidate = random_neighbor(current, vehicles, ship, rng)`
`candidate_value = evaluate(candidate, vehicles, ship)`
`dE = candidate_value - current_value`
 	
`if dE >= 0:`
`  # Sama atau lebih baik: selalu diterima`
`  p = 1.0`
`  current = candidate`
`  current_value = candidate_value`
`else:`
`  # Lebih buruk: diterima dengan peluang e^(dE/T)`
`  p = math.exp(dE / T)`
`  if rng.random() < p:`
`  current = candidate`
`  current_value = candidate_value`
`  accepted_worse = accepted_worse + 1`
`  else:`
`  stuck_count = stuck_count + 1`
	Pada setiap iterasi, pencarian berhenti bila suhu sudah di bawah T_min. Jika belum, satu neighbor acak diambil dengan random_neighbor dan nilainya dihitung. dE adalah selisih nilai neighbor dan nilai state sekarang.
	
	Jika dE ≥ 0 (sama atau lebih baik), neighbor selalu diterima dan peluang p bernilai 1. Jika dE < 0, peluangnya p = e^(dE/T). Neighbor diterima bila angka acak dari rng.random() lebih kecil dari p,

	dan accepted_worse bertambah. Jika ditolak, stuck_count bertambah.

`# Simpan kalau ini state terbaik sejauh ini`
`if current_value > best_value:`
`best = copy_state(current)`
`best_value = current_value`
`state_history.append(copy_state(best))`
`state_history_iter.append(it)`
 	
`accept_prob_history.append(p)`
`history.append(best_value)`
`history_current.append(current_value)`
 	
`T = T * alpha # dinginkan`
`temperature_history.append(T)`
`iterations = it`
	Jika nilai state sekarang melampaui nilai terbaik, state itu disalin ke best dan dicatat di state_history beserta nomor iterasinya. Setelah itu p, nilai terbaik, dan nilai state sekarang dicatat. Terakhir suhu diturunkan: T dikalikan alpha.

`duration = time.perf_counter() - start`
 	
`if iterations > 0:`
`stuck_frequency = stuck_count / iterations`
`else:`
`stuck_frequency = 0.0`
 	
`return {`
`"initial_state": initial_state,`
`"final_state": best,`
`"history": history,`
`"duration": duration,`
`"metrics": {`
`"algorithm": "simulated_annealing",`
`"final_value": best_value,`
`"iterations": iterations,`
`"T0": T0,`
`"alpha": alpha,`
`"T_min": T_min,`
`"T_final": T,`
`"stuck_count": stuck_count,`
`"stuck_frequency": stuck_frequency,`
`"accepted_worse": accepted_worse,`
`"max_iter": max_iter,`
`"seed": seed,`
`},`
`# List yang panjang disimpan di luar "metrics" supaya print_report`
`# tidak membanjiri terminal dengan ribuan angka.`
`"accept_prob_history": accept_prob_history,`
`"history_current": history_current,`
`"temperature_history": temperature_history,`
`"state_history": state_history,`
`"state_history_iter": state_history_iter,`
`}`
	Setelah loop selesai, frekuensi stuck dihitung sebagai stuck_count dibagi jumlah iterasi. Hasilnya berupa kamus berisi initial_state, final_state (state terbaik, bukan state terakhir), history, duration, dan metrics (nilai akhir, jumlah iterasi, T0, alpha, T_min, T_final, stuck_count, stuck_frequency, accepted_worse, max_iter, seed).
	
	Bentuk ini sama dengan algoritma lain, sehingga modul yang membuat grafik dan gambar bisa memakainya tanpa dibedakan. Tiga list yang panjang (accept_prob_history, history_current, temperature_history) ditaruh di luar metrics supaya print_report tidak mencetak puluhan ribu angka.

2.3.3. Hasil Eksperimen

SA dijalankan 3 kali pada demo.json dengan seed 1, 2, dan 3. Seed yang sama dipakai oleh Hill Climbing, sehingga state awal tiap seed sama. Parameter yang dipakai: T0 otomatis (86,27), α otomatis (0,999662), T_min = 0,1, max_iter = 20.000, dan p0 = 0,8. State awal, state akhir, dan grafik tiap run ditampilkan pada Gambar 2.3.3.1 sampai 2.3.3.9.

Tabel 2.3.3.1 Hasil Eksperimen Simulated Annealing

Seed	Durasi (detik)	Nilai Awal	Nilai Akhir	Iterasi	Suhu akhir
1	2,567	95	187	20.000	0,1
2	2,381	-1297	189	20.000	0,1
3	2,620	-7199	189	20.000	0,1

Rincian state akhir dari percobaan adalah sebagai berikut.

Tabel 2.3.3.2 Rincian State Akhir Simulated Annealing

Seed	Kendaraan Terangkut	Total Shipping Fee	Total Berat (maksimal 120)	Kendaraan di luar kapal	Validity
1	9	187	119	B, C, G	Ya
2	9	189	120	B, C, L	Ya
3	9	189	120	B, C, L	Ya

Ringkasan tiga percobaan: nilai akhir rata-rata 188,33 (terendah 187, tertinggi 189, selisih 2), dan durasi rata-rata 2,52 detik.

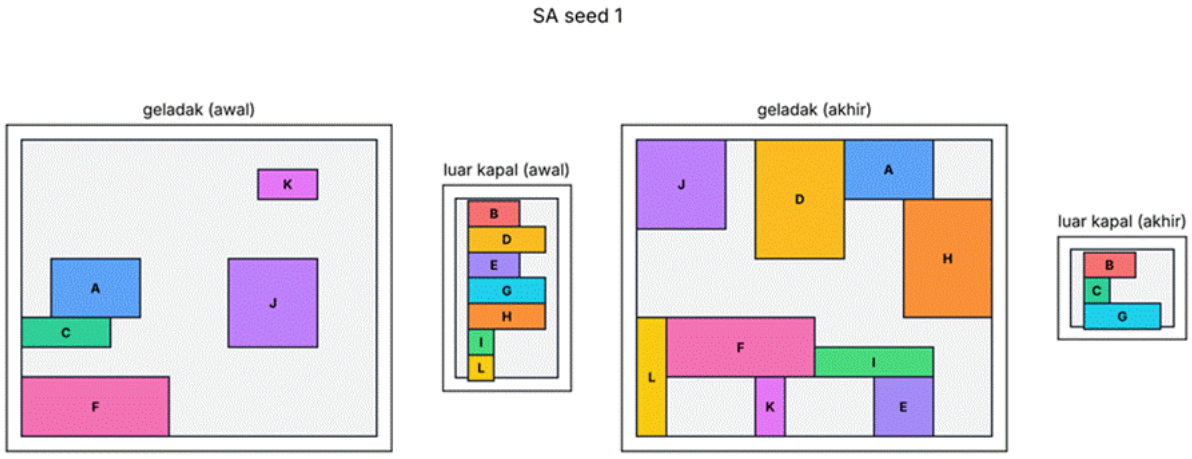
Tabel 2.3.3.3 Metrik Simulated Annealing

Seed	T0	α	T akhir	Stuck	Frekuensi stuck	Langkah buruk diterima
1	86,27	0,999662	0,1	13.441	0,6721	577
2	86,27	0,999662	0,1	13.349	0,6674	579
3	86,27	0,999662	0,1	13.362	0,6681	580

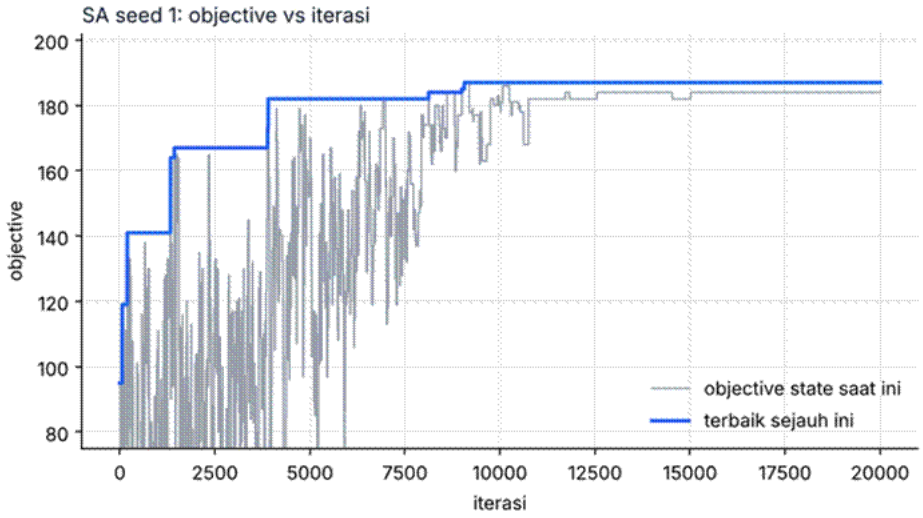
Tabel 2.3.3.4 Perkembangan Pencarian Simulated Annealing

Seed	Nilai pada iterasi 1.000	Pertama kali mencapai ≥ 180 (iterasi, suhu)	Nilai terbaik ditemukan (iterasi, suhu)

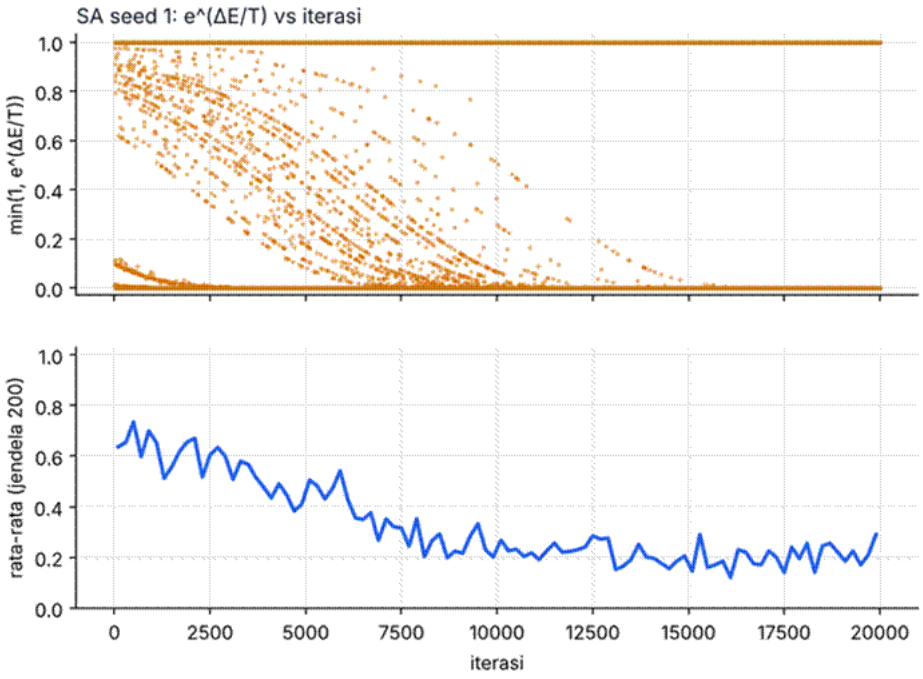
1	83	3.903 (T = 23,06)	9.062 (T = 4,03)
2	53	4.578 (T = 18,36)	12.376 (T = 1,32)
3	80	5.863 (T = 11,89)	9.235 (T = 3,80)



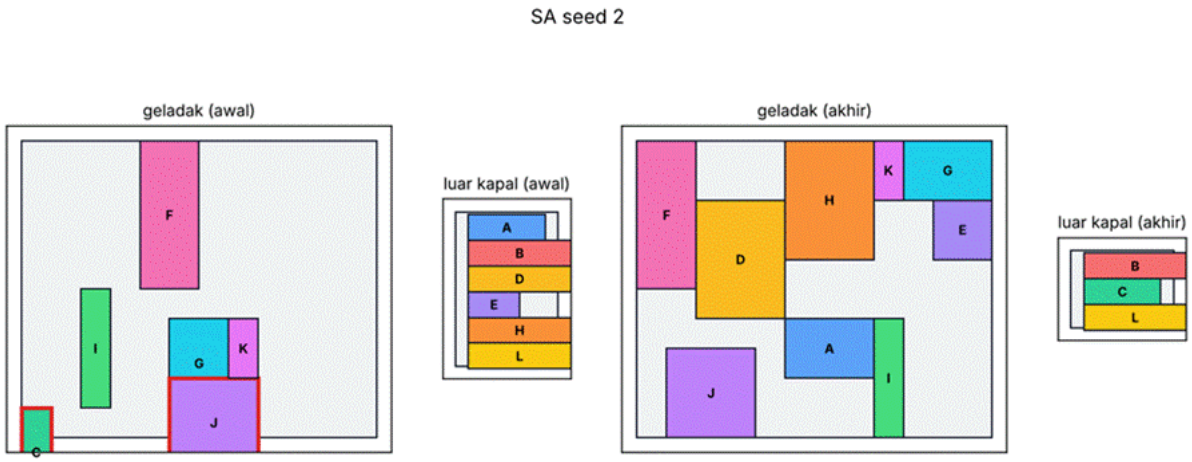
Gambar 2.3.3.1 State awal dan state akhir Simulated Annealing, seed 1



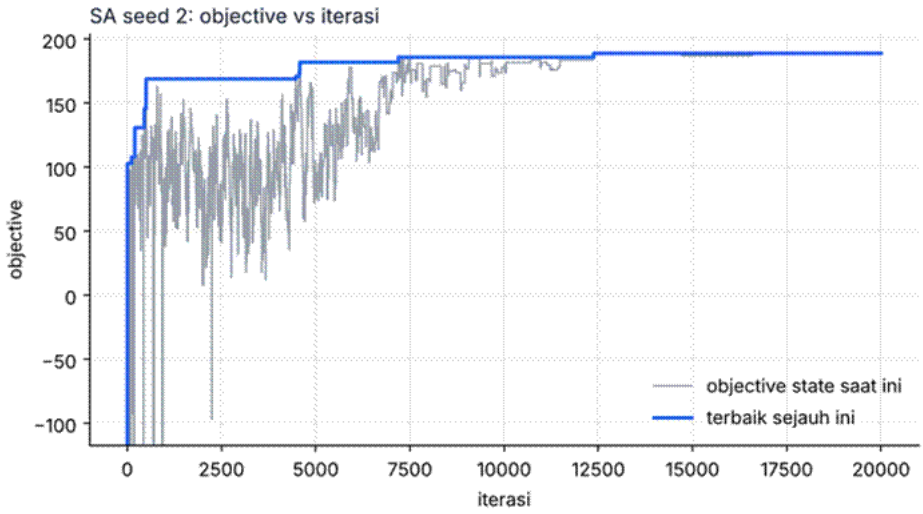
Gambar 2.3.3.2 Objective terhadap iterasi, Simulated Annealing seed 1



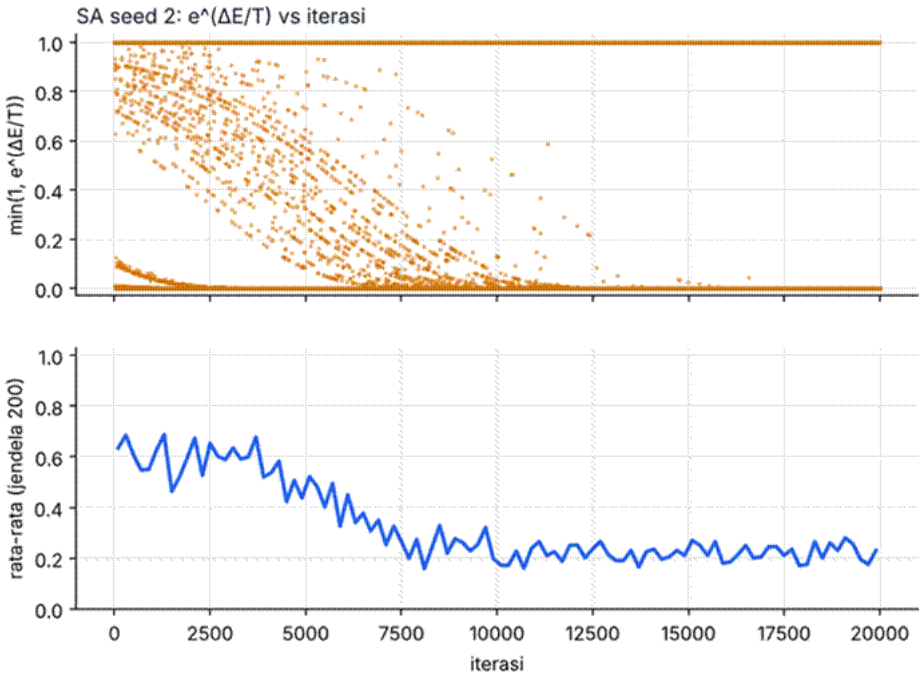
Gambar 2.3.3.3 Peluang penerimaan min(1, e^(ΔE/T)) terhadap iterasi, Simulated Annealing seed 1



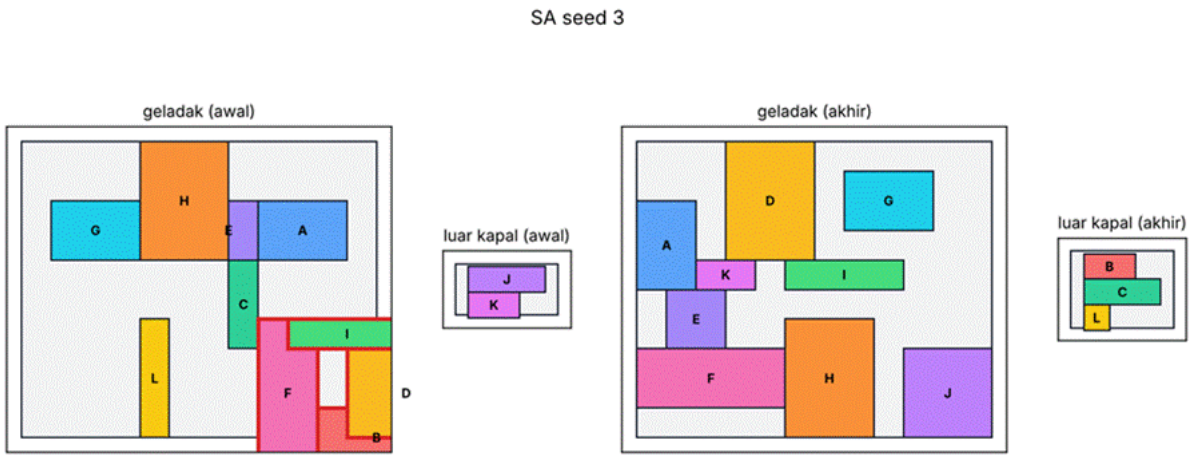
Gambar 2.3.3.4 State awal dan state akhir Simulated Annealing, seed 2



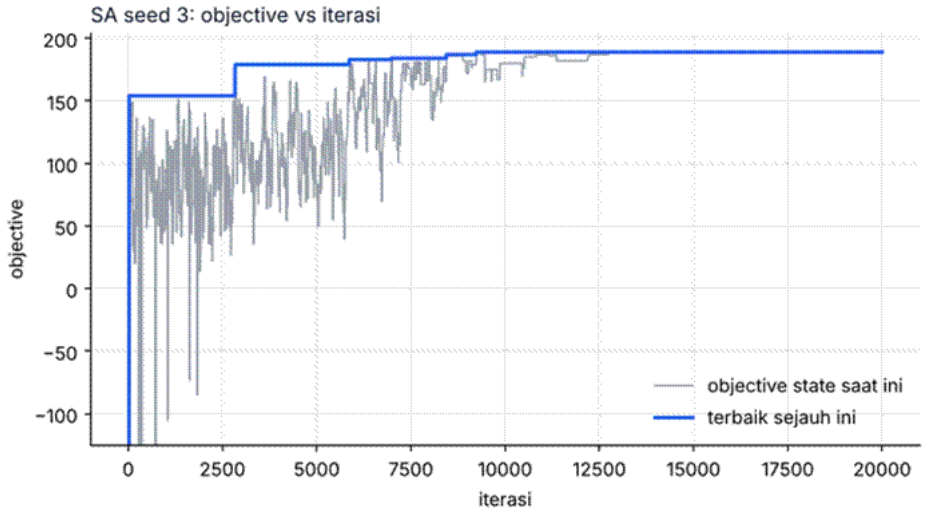
Gambar 2.3.3.5 Objective terhadap iterasi, Simulated Annealing seed 2



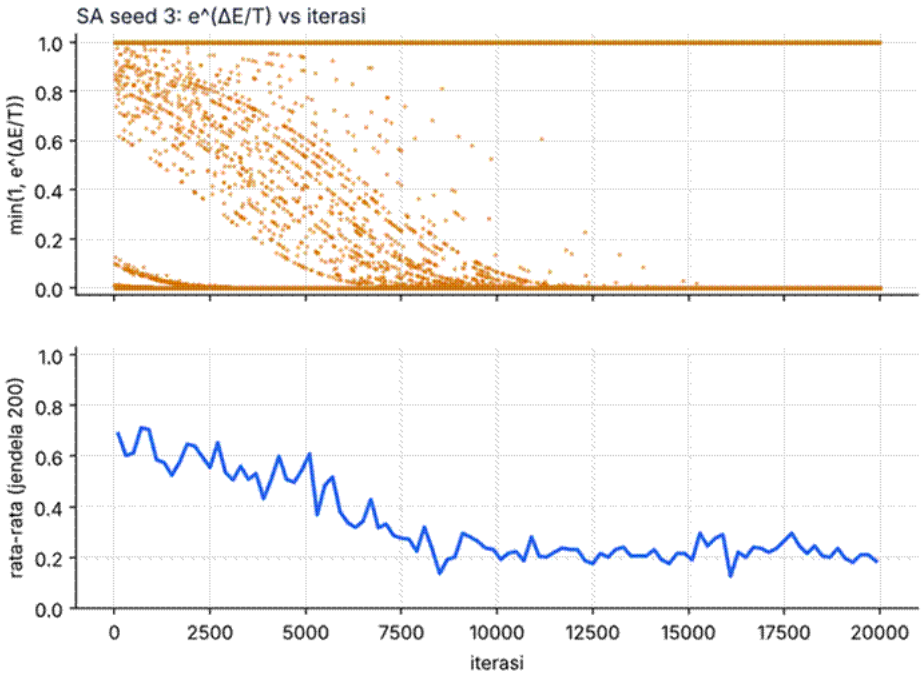
Gambar 2.3.3.6 Peluang penerimaan min(1, e^(ΔE/T)) terhadap iterasi, Simulated Annealing seed 2



Gambar 2.3.3.7 State awal dan state akhir Simulated Annealing, seed 3



Gambar 2.3.3.8 Objective terhadap iterasi, Simulated Annealing seed 3



Gambar 2.3.3.9 Peluang penerimaan min(1, e^(ΔE/T)) terhadap iterasi, Simulated Annealing seed 3

2.3.4. Pembahasan Hasil

Nilai akhir SA adalah 187, 189, dan 189. Dua dari tiga run mencapai 189, yaitu batas atas dari perhitungan knapsack pada bagian 2.1.5. Karena state-nya sah, 189 adalah optimum global untuk demo.json. Seed 1 berhenti di 187, selisih 2 dari optimum.

Titik awal hampir tidak berpengaruh. Nilai awal ketiga run sangat berbeda (95, −1297, dan −7199), tetapi pada iterasi 1.000 nilainya sudah 83, 53, dan 80, dan semuanya sampai di 180 ke atas antara iterasi 3.900 dan 5.900. State awal dengan penalti berat pulih dengan cepat karena langkah yang mengeluarkan kendaraan bermasalah menaikkan nilai sampai ribuan.

Penerimaan langkah buruk hampir seluruhnya terjadi di paruh pertama pencarian. Dari sekitar 580 langkah buruk yang diterima tiap run (2,9% dari 20.000 iterasi), 569 sampai 577 terjadi sebelum iterasi 10.000, saat suhu masih di atas sekitar 3. Pada 30% iterasi terakhir hampir tidak ada langkah buruk yang diterima. Pola ini terlihat di grafik peluang: rata-rata min(1, e^(ΔE/T)) per 2.000 iterasi turun dari 0,59 sampai 0,63 di awal menjadi 0,21 sampai 0,23 di akhir. Karena langkah yang tidak memperburuk nilai selalu mendapat peluang 1, angka akhir itu kira-kira sama dengan porsi neighbor dengan $\Delta E \geq 0$. Artinya sekitar seperlima neighbor

yang dicoba di akhir pencarian tidak menurunkan nilai, dan sisanya hampir pasti ditolak.

Frekuensi stuck berada di 0,667 sampai 0,672, jadi dua pertiga iterasi berakhir dengan penolakan kandidat yang lebih buruk. Angka ini tinggi karena kebanyakan neighbor acak memang memperburuk nilai: memindahkan kendaraan ke petak yang sudah terisi menambah penalti 200 per petak, dan mengeluarkan kendaraan menurunkan fee. Jadi stuck di sini tidak berarti pencarian macet. Ia hanya mencatat bahwa state sekarang bertahan karena kandidatnya ditolak.

Nilai terbaik ditemukan di iterasi 9.062 sampai 12.376, pada suhu 1,3 sampai 4,0. Setelah itu tidak ada perbaikan, padahal masih ada sekitar 38% sampai 55% iterasi tersisa. Pada tiga run ini, kenaikan nilai terjadi di rentang suhu menengah. Suhu awal yang tinggi dipakai untuk menjelajah, dan suhu di bawah sekitar 1,3 tidak menambah apa-apa pada ketiga run. Kami hanya menguji satu pasang T0 dan α, jadi pengaruh keduanya terhadap hasil belum bisa dipastikan. Jumlah run juga hanya tiga, sehingga kesimpulan di atas perlu dibaca sebagai gambaran pada data uji ini saja.

2.4. Genetic Algorithm

2.4.1. Cara Kerja Umum

Genetic Algorithm merupakan teknik pencarian dan optimasi berbasis heuristik yang mengadaptasi mekanisme evolusi biologis serta seleksi alam (Holland, 1975). Metode ini bekerja dengan mengelola sekumpulan kandidat solusi yang direpresentasikan sebagai populasi kromosom. Setiap solusi dievaluasi menggunakan fitness function untuk mengukur tingkat optimalnya terhadap permasalahan yang diselesaikan. Proses pencarian solusi dalam Algoritma Genetika dilakukan melalui tiga proses utama:

1. Seleksi (Selection), yaitu memilih individu dengan nilai fitness tinggi untuk dijadikan parent.
2. Persilangan (Crossover), yaitu menggabungkan materi genetik dari dua parent untuk menghasilkan kandidat solusi baru.
3. Mutasi (Mutation), yaitu mengubah sebagian kecil nilai gen secara acak untuk mempertahankan variasi populasi dan mencegah algoritma terjebak pada solusi local optima.

2.4.2. Source Code

Pada persoalan ini, individu atau kromosom diibaratkan dengan satu state konfigurasi kendaraan dalam kapal. Digunakan juga disini istilah populasi, *fitness function*, generasi (jumlah iterasi), dan *mutation rate*. Berikut merupakan penjelasan masing-masing fungsi yang digunakan dalam *genetic algorithm*.

1	```
	def initiate_pop(vehicles, ship, pop_size):
	    populasi = []
	    for i in range(pop_size):
	        individu = random_state(vehicles, ship)
	        populasi.append(individu)
	    return populasi
	``
		<u>Fungsi initiate_pop()</u>
		Fungsi untuk menginisiasi populasi. Fungsi ini membuat array populasi kosong, kemudian membuat individu sebanyak pop_size dan memasukkannya ke array populasi, tiap individu adalah state acak yang dihasilkan oleh random_state dari modul state

2	```
	def selection(populasi, scores):
	    # normalisasi semua skor ke positif
	    skor_terburuk = min(scores)
	    fitness = []
	    for skor in scores:
	        nilai = skor - skor_terburuk + 1
	        fitness.append(nilai)
	
	    # sum total fitness, hitung persen tiap individu
	    total = 0
	    for nilai in fitness:
	        total = total + nilai
	    persen = []
	    for nilai in fitness:
	        persen.append(nilai / total * 100)
	
	    # lempar dadu 0-100
	    dadu = random.uniform(0, 100)
	``
		<u>Fungsi selection()</u>
		Proses seleksi menggunakan metode roulette wheel selection. Karena skor objective bisa bernilai negatif (karena penalty), seluruh skor akan dinormalisasi menjadi positif dengan menggeser setiap skor sebesar selisihnya terhadap skor terburuk ditambah 1.
		
		Setiap individu kemudian diberikan probabilitas terpilih yang sebanding dengan fitness nya.
		
		Kemudian pemilihan parent dilakukan dengan melakukan random 0-100 terhadap

	<pre>batas = 0
	for i in range(len(persen)):
	batas = batas + persen[i]
	if dadu <= batas:
	return populasi[i]
	
	# fallback (klo dadu meleset dikit karna pembulatan desimal)
	return populasi[len(populasi) - 1]</pre>
		distribusi kumulatif persentase fitness, dan populasi yang terpilih direturn.
		
		Ada juga mekanisme fallback yang mengembalikan individu terakhir bila terjadi kesalahan pembulatan desimal (misal total batas kumulatif 99.99 dan hasil dadunya 100).

3	<pre>def crossover(parent_a, parent_b):
	n = len(parent_a)
	titik = random.randint(1, n - 1)
	
	child_1 = []
	child_2 = []
	for i in range(n):
	if i < titik:
	
	child_1.append(dict(parent_a[i]))
	
	child_2.append(dict(parent_b[i]))
	else:
	
	child_1.append(dict(parent_b[i]))
	
	child_2.append(dict(parent_a[i]))
	return child_1, child_2</pre>
		<u>Fungsi crossover()</u>
		Menggunakan single-point crossover, yaitu dipilih satu titik potong acak kemudian child pertama mewarisi gen parent A sebelum titik potong dan gen parent B sesudahnya (dan sebaliknya untuk child satunya). Setiap gen di-copy dengan dict() agar child tidak berbagi referensi memori dengan parent

4	<pre>def mutasi(state, ship, mutation_rate): # ubah 1 kendaraan random sesuai mutation_rate # pindah posisi/putar orientasi/keluar-masuk kapal/tukar posisi 2 kendaraan if random.random() >=</pre>	<u>Fungsi mutasi()</u> Mutasi terjadi dengan probabilitas sebesar mutation_rate. Apabila terjadi, maka dipilih satu kendaraan secara acak lalu diterapkan satu dari empat operator move secara acak: move_relocate (pindah posisi), move_rotate (ubah orientasi), move_toggle (keluar/masuk

	mutation_rate:
	return state
	i = random.randint(0, len(state) - 1)
	pilihan = random.randint(0, 3)
	if pilihan == 0:
	return move_relocate(state, i, ship)
	elif pilihan == 1:
	return move_rotate(state, i)
	elif pilihan == 2:
	return move_toggle(state, i)
	else:
	# swap: tukar posisi & orientasi kendaraan i dengan kendaraan random lain
	j = random.randint(0, len(state) - 1)
	while j == i:
	j = random.randint(0, len(state) - 1)
	return move_swap(state, i, j)
		kapal), atau move_swap (swap dengan kendaraan lain)

5	def genetic_algorithm(vehicles, ship, pop_size=50, generations=100, mutation_rate=0.2):
	start = time.time()
	
	populasi = initiate_pop(vehicles, ship, pop_size)
	
	# cari individu terbaik di populasi awal
	best = populasi[0]
	best_score = evaluate(best, vehicles, ship)
	for individu in populasi:
		Fungsi genetic_algorithm()
		Fungsi utama genetic algorithm.
		
		Dimulai dengan inisialisasi populasi awal dengan memanggil fungsi initiate_pop()
		
		Kemudian dilakukan evaluasi awal untuk menentukan individu terbaik sebagai initial state
		
		Kemudian dilakukan iterasi sebanyak

```
        score = evaluate(individu,
vehicles, ship)
        if score > best_score:
            best = individu
            best_score = score

    initial_state = best

    history = [best_score]        #
objective terbaik per generasi
    history_avg = [best_score]    #
rata-rata objective populasi per
generasi
    state_history = [copy(best)]  #
state terbaik per generasi

    for generasi in range(generations):
        # hitung score semua individu
sekali (utk roulette & rata-rata)
        scores = []
        for individu in populasi:

scores.append(evaluate(individu,
vehicles, ship))

# elitisme 2 individu terbaik langsung
ikut ke generasi berikutnya
        # ref:
https://www.tutorialspoint.com/genetic_a
lgorithms/genetic_algorithms_survivor_se
lection.htm
        idx_best_first = 0
        idx_best_second = 1
        if scores[idx_best_second] >
scores[idx_best_first]:
            idx_best_first = 1
            idx_best_second = 0
        for i in range(2,
len(populasi)):
```

generations

Untuk setiap generasi, skor seluruh populasi dihitung sekali, lalu populasi baru dibuat lewat seleksi, crossover, dan mutasi hingga penuh, kemudian best score dan rata-rata skor populasi dicatat ke history. Dilakukan juga elitism (ref) dengan langsung mengikutkan 2 individu terbaik ke generasi berikutnya supaya crossover dan mutasi yang bersifat acak tidak merusak solusi optimal yang sudah didapat di generasi sebelumnya

Untuk history yang dicatat adalah history best score objective function, average score objective function, dan best state di tiap generasi

Struktur output fungsi genetic algorithm terdiri dari initial state, final state, history nilai objective terbaik dan rata-rata, durasi, metrik parameter, dan history state (untuk visualisasi saja)

```
            if scores[i] >
scores[idx_best_first]:
                idx_best_second =
idx_best_first
                idx_best_first = i
            elif scores[i] >
scores[idx_best_second]:
                idx_best_second = i

        populasi_baru =
[copy(populasi[idx_best_first]),
copy(populasi[idx_best_second])]



        while len(populasi_baru) <
pop_size:
            parent_a =
selection(populasi, scores)
            parent_b =
selection(populasi, scores)
            child_1, child_2 =
crossover(parent_a, parent_b)
            child_1 = mutasi(child_1,
ship, mutation_rate)
            child_2 = mutasi(child_2,
ship, mutation_rate)

populasi_baru.append(child_1)
            if len(populasi_baru) <
pop_size:  # biar ga overload popsize

populasi_baru.append(child_2)

        populasi = populasi_baru

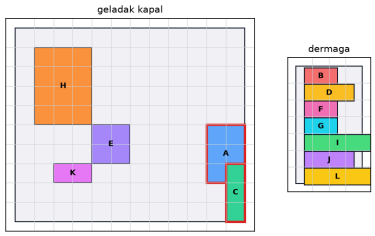
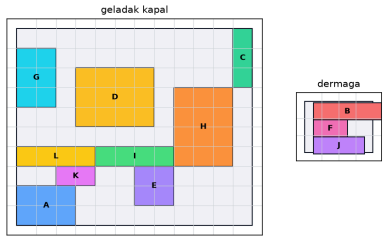
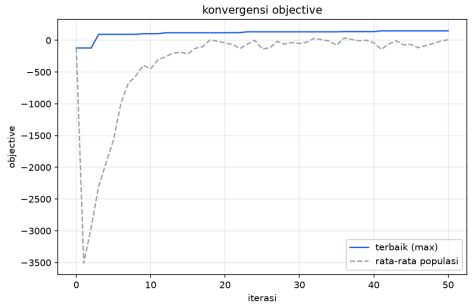
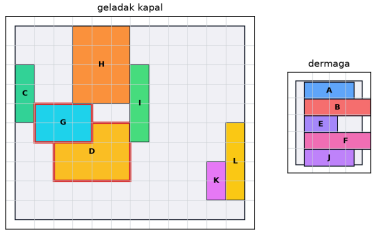
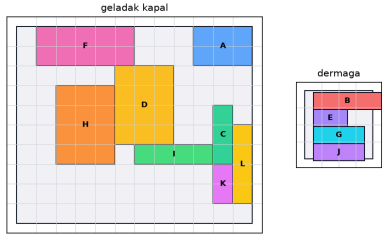
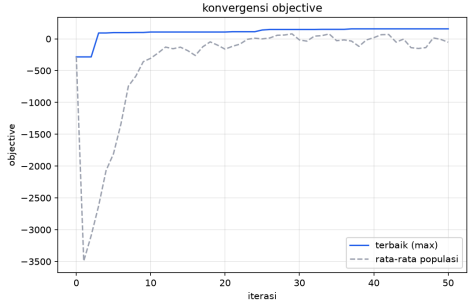
        # update best + catat history
(max & avg populasi)
        total = 0
        for individu in populasi:
```

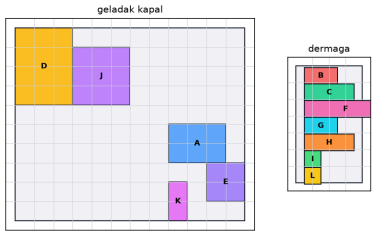
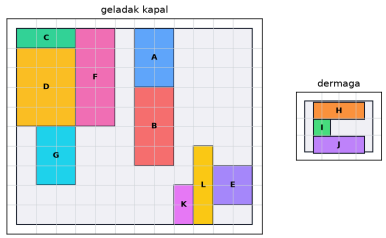
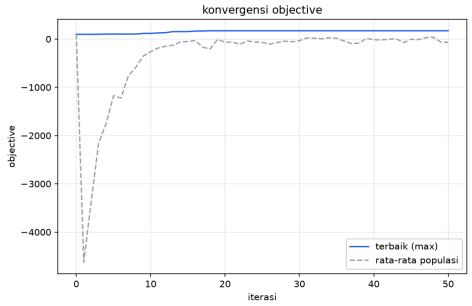
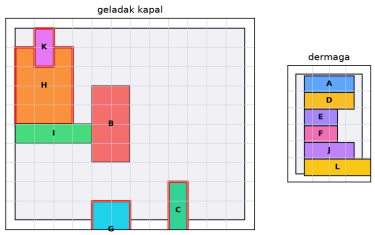
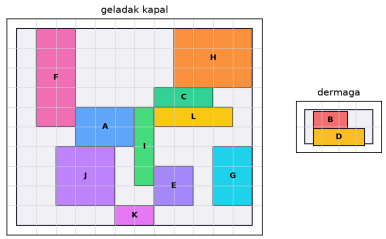
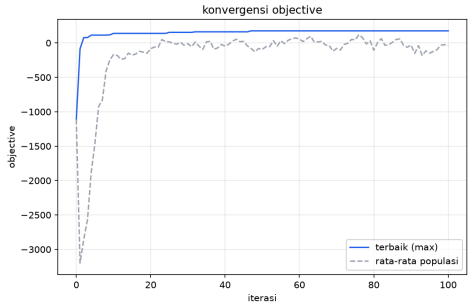
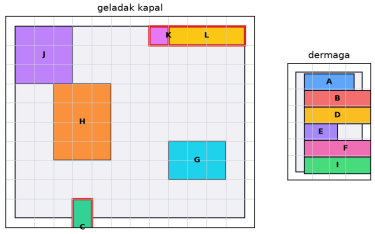
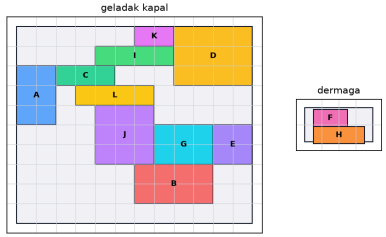
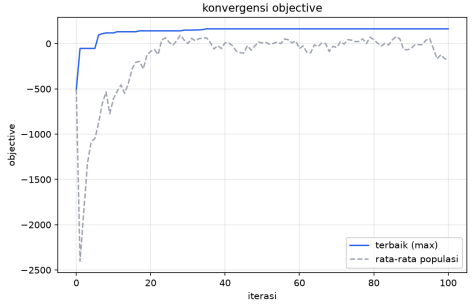
	<pre>score = evaluate(individu, vehicles, ship)
	total = total + score
	if score > best_score:
	best = individu
	best_score = score
	rata_rata = total / len(populasi)
	
	history.append(best_score)
	history_avg.append(rata_rata)
	state_history.append(copy(best))
	
	return {
	"initial_state": initial_state,
	"final_state": best,
	"history": history,
	"duration": time.time() - start,
	"metrics": {
	"pop_size": pop_size,
	"generations": generations,
	"mutation_rate": mutation_rate,
	"best_score": best_score,
	},
	"history_avg": history_avg,
	"state_history": state_history
	}</pre>	

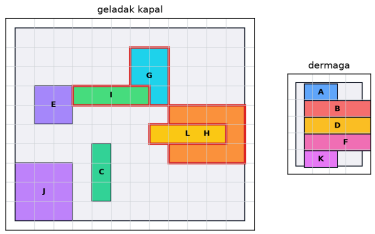
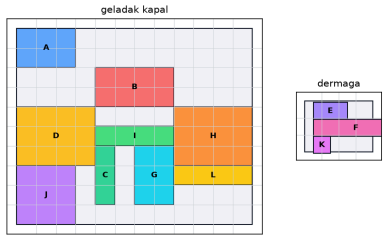
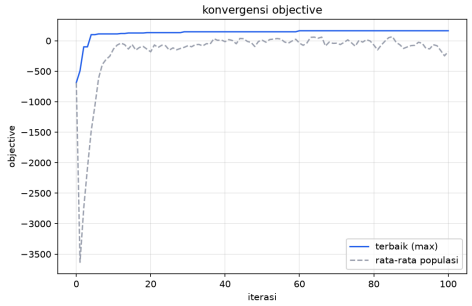
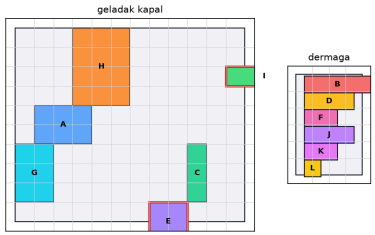
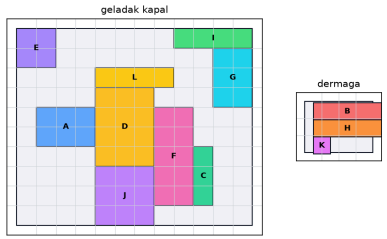
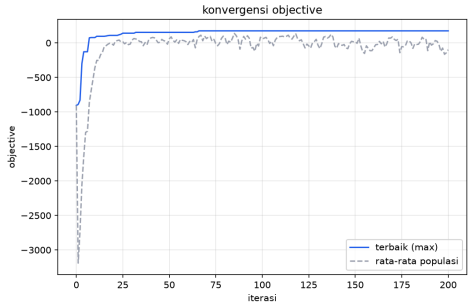
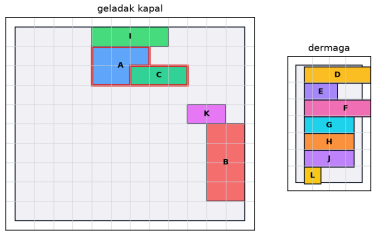
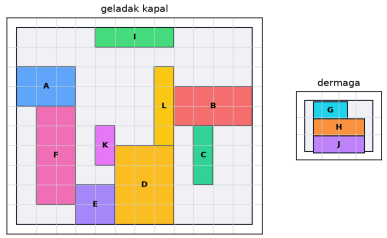
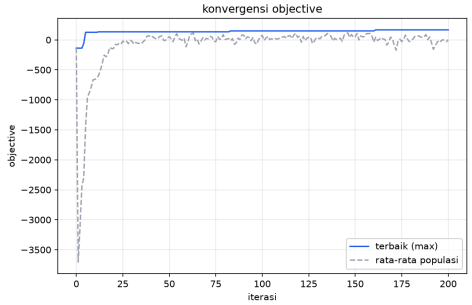
6	<pre>def copy(state): hasil_copy = [] for kendaraan in state: hasil_copy.append(dict(kendaraan)) return hasil_copy</pre>	Fungsi copy() Fungsi bantu deep copy sederhana. Digunakan untuk menyimpan snapshot state terbaik tiap generasi

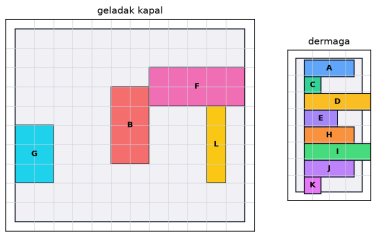
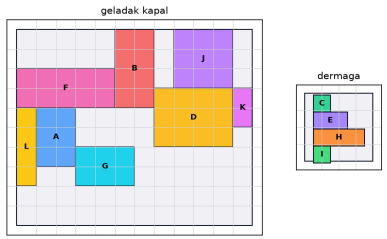
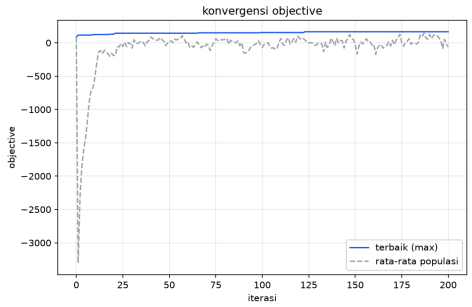
2.4.3. Hasil Eksperimen

Skenario A (pop_size variabel kontrol)

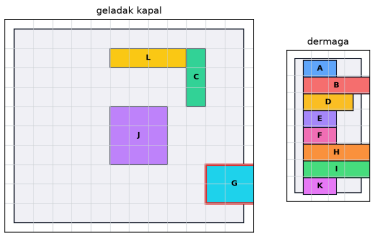
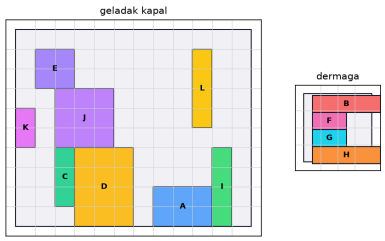
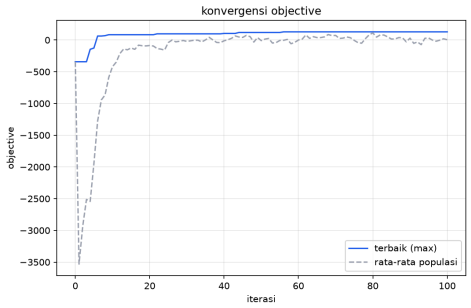
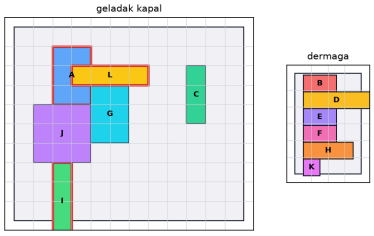
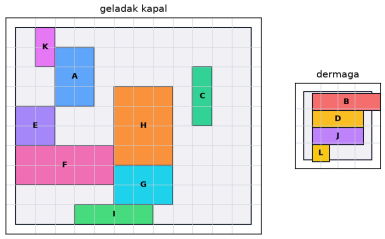
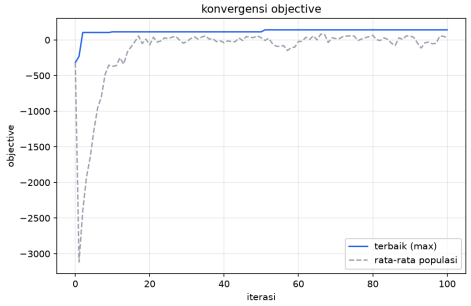
No	Jumlah Populasi	Banyak Iterasi	State Awal	State Akhir	Durasi	Objective Score	Plot Objective Function
1a	50	50			0.43353 4622192 3828	first=-121 , last=149	
1b	50	50			0.43162 2505187 9883	first=-284 , last=157	

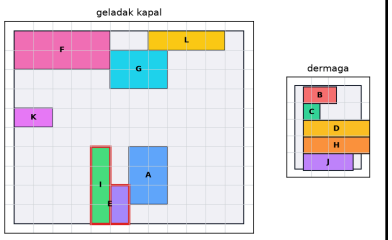
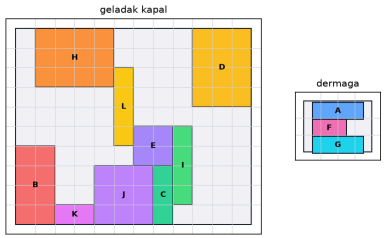
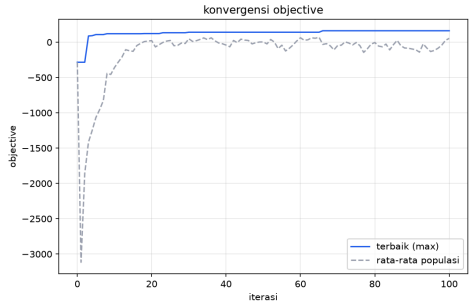
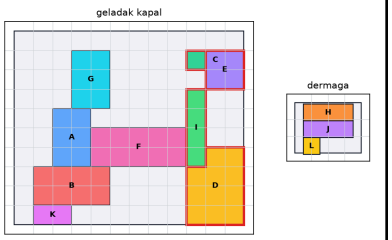
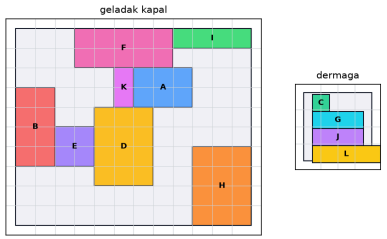
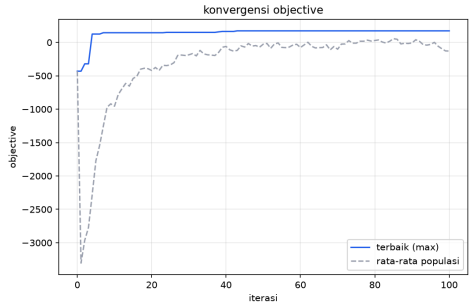
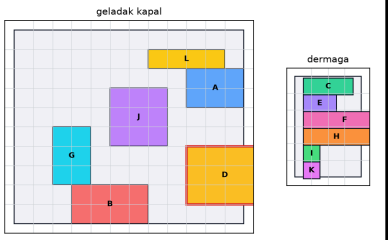
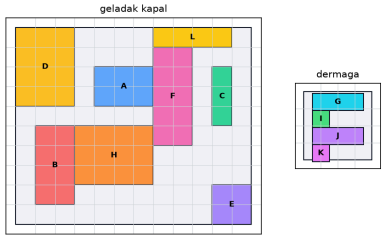
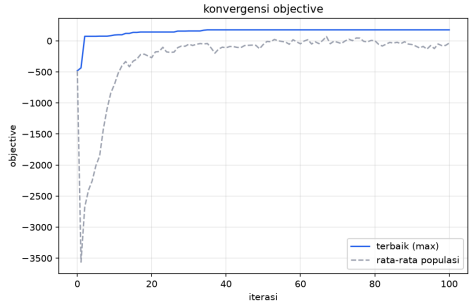
1c	50	50			0.42248 4397888 1836	first=98, last=172	
2a	50	100			1.20956 6831588 7451	first=-111 1, last=176	
2b	50	100			1.17531 5141677 8564	first=-506 , last=163	

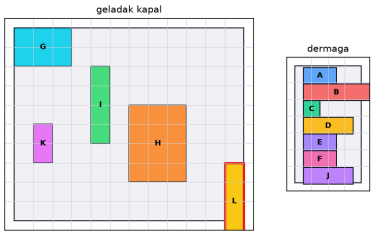
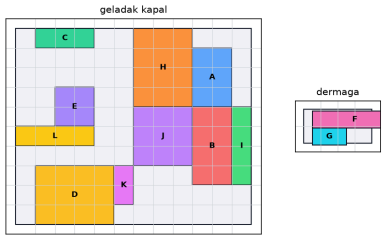
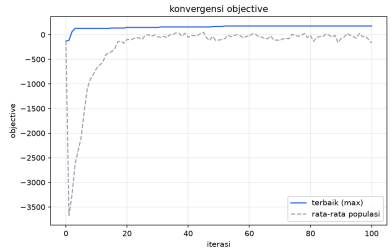
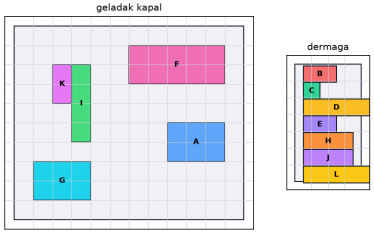
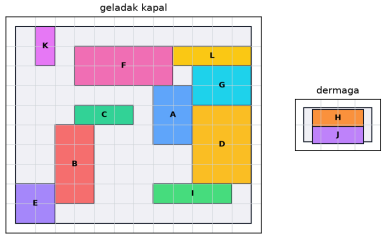
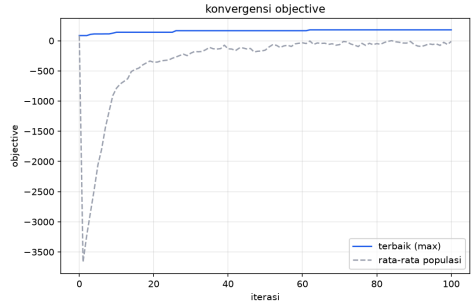
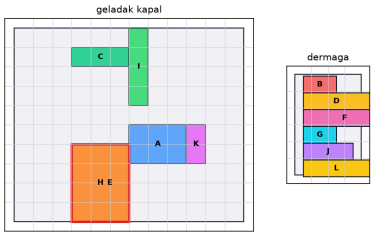
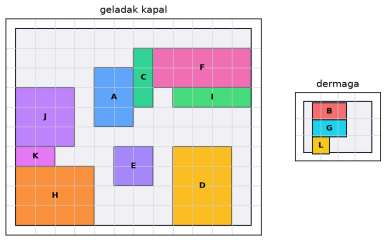
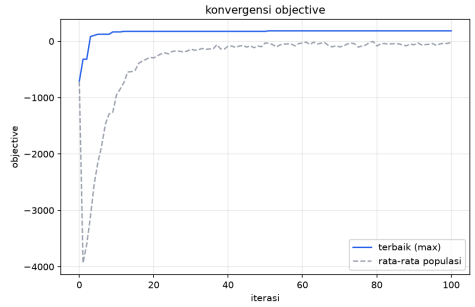
2c	50	100			1.171956539154 0527	first=-687 , last=165	
3a	50	200			2.401986122131 3477	first=-906 , last=175	
3b	50	200			2.150836467742 92	first=-138 , last=167	

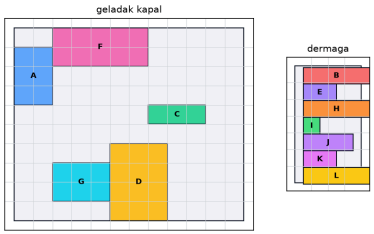
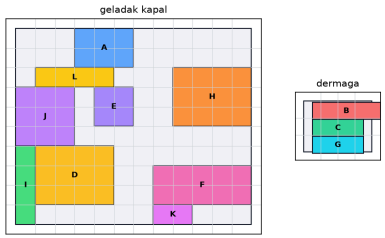
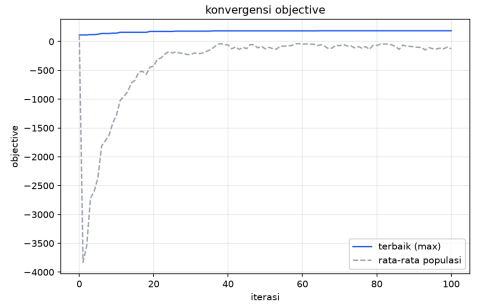
3c	50	200			2.04302 8354644 7754	first=86, last=166	

Skenario B (generations variabel kontrol)

No	Jumlah Populasi	Banyak Iterasi	State Awal	State Akhir	Durasi	Objective Score	Plot Objective Function
1a	50	100			0.93051 1951446 5332	first=-342 , last=129	
1b	50	100			1.01010 4417800 9033	first=-318 , last=142	

1c	50	100			1.04572 7252960 205	first=-284 , last=162	
2a	100	100			1.70160 1028442 3828	first=-431 , last=173	
2b	100	100			1.70195 2695846 5576	first=-482 , last=178	

2c	100	100			2.20830 3213119 507	first=-129 , last=177	
3a	200	100			3.36607 4800491 333	first=86, last=181	
3b	200	100			3.44651 6036987 3047	first=-712 , last=185	

3c	200	100			4.30420 5179214 4775	first=114, last=187	

2.4.4. Pembahasan Hasil

Pada skenario A, jumlah populasi dikontrol sebesar 50 individu, sedangkan jumlah iterasi divariasikan sebanyak 50, 100, dan 200 iterasi dengan masing-masing dijalankan 3 kali. Hasil eksperimen menunjukkan bahwa penambahan banyak iterasi meningkatkan nilai objective function akhir, meskipun peningkatannya semakin kecil. Pada 50 iterasi, rata-rata nilai objective akhir adalah 159. Pada 100 iterasi, rata-rata nilai objective akhirnya sebesar 168. Sedangkan pada 200 iterasi, rata-rata objective akhir sebesar 169. Tren tersebut sejalan juga dengan bentuk kurva pada plot konvergensi, di mana nilai objective terbaik meningkat tajam pada 10 hingga 20 iterasi pertama, kemudian melandai dan hampir mendatar. Jika dilihat nilai objective awal dan akhirnya, didapat bahwa algoritma dapat mencapai local optimum yang serupa dari titik awal mana pun meskipun belum dapat dipastikan bahwa region tersebut merupakan global optimum. Secara durasi, waktu pencarian kurang lebih meningkat terhadap banyak iterasi, yaitu sekitar 0.42-0.43 detik untuk 50 iterasi, 1.17-1.21 detik untuk 100 iterasi, dan 2.04-2.40 detik untuk 200 iterasi.

Pada skenario B, banyak iterasi dikontrol sebesar 100, jumlah populasinya divariasi sebesar 50, 100, dan 200 individu. Hasil eksperimen menunjukkan bahwa jumlah populasi memberikan pengaruh yang lebih kuat terhadap score objective function dibandingkan banyak iterasi. Populasi sebesar 50 menghasilkan rata-rata objective akhir sekitar 144. Populasi 100 menghasilkan rata-rata sekitar 176. Sedangkan populasi 200 rata-rata sekitar 184. Populasi yang lebih besar juga memberikan hasil yang lebih konsisten. Pada skenario ini durasi pencarian meningkat signifikan, dari sekitar 0.93-1.05 detik pada populasi 50 menjadi 1.70-2.21 detik pada populasi 100 dan 3.37-4.30 detik pada populasi 200, karena jumlah evaluasi fitness pada tiap generasi sebanding dengan ukuran populasi.

Secara keseluruhan, baik banyak iterasi maupun jumlah populasi berpengaruh positif terhadap nilai objective akhir. Namun perlu dicatat bahwa penambahan

iterasi dari 100 ke 200 tidak menaikkan rata-rata score objective secara signifikan. Penambahan jumlah populasi dari 50 ke 200 memberikan peningkatan rata-rata score objective sekitar 40 poin sekaligus stabilitas hasil yang jauh lebih baik.

2.5. Analisis Perbandingan

Data Hill Climbing (sideways) dan Simulated Annealing berasal dari tiga run (seed 1, 2, 3) pada demo.json. Data Genetic Algorithm diambil dari bagian 2.4.3: skenario A (populasi 50, generasi 50, 100, dan 200) dan skenario B (generasi 100, populasi 50, 100, dan 200). Konfigurasi populasi 50 dengan 100 generasi muncul di kedua skenario sehingga tercatat dua kali. Simpangan baku dihitung sebagai simpangan baku sampel ($n - 1$), dan semua angka dibulatkan ke dua desimal.

2.5.1. Ringkasan Lintas Algoritma

Tabel 2.5.1.1 Ringkasan Hasil per Algoritma

Algoritma	Konfigurasi	Objective akhir (run 1, 2, 3)	Rata-rata	Simpangan baku
Hill Climbing (sideways)	max_sideways 100, k_neighbors 200	184, 189, 181	184,67	4,04
Simulated Annealing	T0 86,27, α 0,999662, 20.000 iterasi	187, 189, 189	188,33	1,15
Genetic Algorithm	populasi 50, 100 generasi (skenario A)	176, 163, 165	168,00	7,00
Genetic Algorithm	populasi 50, 100 generasi (skenario B)	129, 142, 162	144,33	16,62
Genetic Algorithm	populasi 100, 100 generasi	173, 178, 177	176,00	2,65
Genetic Algorithm	populasi 200, 100 generasi	181, 185, 187	184,33	3,06

Urutan berdasarkan rata-rata adalah SA (188,33), HC (184,67), GA populasi 200 (184,33), GA populasi 100 (176,00), lalu GA populasi 50 (168,00 dan 144,33 pada dua set run).

2.5.2. Kedekatan ke Global Optimum

Batas atas dari bagian 2.1.5 adalah 189. HC dan SA masing-masing menemukan state sah bernilai 189, sehingga batas itu tercapai dan 189 adalah optimum global demo.json.

Tabel 2.5.2.1 Selisih terhadap Optimum (189)

Algoritma	Rata-rata	Selisih rata-rata	Selisih (%)	Run yang mencapai 189
Hill Climbing (sideways)	184,67	4,33	2,29	1 dari 3
Simulated Annealing	188,33	0,67	0,35	2 dari 3
GA, populasi 200, 100 generasi	184,33	4,67	2,47	0 dari 3
GA, populasi 100, 100 generasi	176,00	13,00	6,88	0 dari 3
GA, populasi 50, 100 generasi (A)	168,00	21,00	11,11	0 dari 3
GA, populasi 50, 100 generasi (B)	144,33	44,67	23,63	0 dari 3

Dari seluruh 18 run GA pada bagian 2.4.3, nilai tertingginya 187 (populasi 200, run 3c), sehingga tidak ada run GA yang mencapai 189. SA paling dekat ke optimum dengan selisih rata-rata 0,35%.

2.5.3. Perbandingan Hasil antar Algoritma

SA menghasilkan nilai rata-rata tertinggi (188,33). HC dengan sideways move (184,67) dan GA dengan populasi 200 (184,33) hampir sama. GA dengan populasi lebih kecil berada di bawahnya, dan GA pada populasi 50 hasilnya paling rendah.

Ada tiga catatan untuk perbandingan ini. Pertama, HC dan SA dimulai dari state awal yang sama untuk tiap seed (95, −1297, dan −7199). State awal GA adalah individu terbaik dari populasi awal, sehingga tidak sama dengan keduanya. Kedua, ketiga algoritma memakai jumlah evaluasi objective yang berbeda (lihat 2.5.4), jadi perbedaan hasil sebagian mencerminkan perbedaan anggaran komputasi. Ketiga, GA dengan

populasi 200 memakai sekitar 40.000 evaluasi per run, dua kali lipat SA (20.000 evaluasi), tetapi nilainya masih di bawah SA.

Pada HC dan SA, state akhir yang bernilai 189 sama persis: kendaraan A, D, E, F, G, H, I, J, dan K diangkut dengan total berat 120 (tepat sebesar kapasitas), sedangkan B, C, dan L tidak diangkut. Itu terjadi pada HC seed 2 serta SA seed 2 dan 3. HC seed 1 (184) dan seed 3 (181) memilih kombinasi kendaraan lain dengan nilai lebih rendah, dan SA seed 1 (187) mengangkut kombinasi yang berbeda dari kombinasi optimum.

2.5.4. Pebandingan Durasi

Tabel 2.5.4.1 Durasi dan Jumlah Evaluasi Objective

Algoritma	Durasi per run (detik)	Evaluasi objective per run
Hill Climbing (sideways)	3,710; 4,695; 2,810	22.200; 31.400; 21.200 (iterasi × 200 neighbor)
Simulated Annealing	2,567; 2,381; 2,620	20.000
GA, populasi 50, 100 generasi	1,17 sampai 1,21 (A); 0,93 sampai 1,05 (B)	sekitar 10.000
GA, populasi 100, 100 generasi	1,70 sampai 2,21	sekitar 20.000
GA, populasi 200, 100 generasi	3,37 sampai 4,30	sekitar 40.000

Jumlah evaluasi GA dihitung sebagai sekitar 2 × populasi × generasi, karena kode GA mengevaluasi seluruh populasi dua kali per generasi (untuk roulette dan untuk memperbarui nilai terbaik). Pada HC, tiap iterasi mengevaluasi 200 neighbor dan pencarian berhenti setelah 106 sampai 157 iterasi. Pada SA, tiap iterasi mengevaluasi satu neighbor selama 20.000 iterasi.

Durasi mengikuti jumlah evaluasi pada ketiga algoritma. Satu evaluasi memakan sekitar 0,12 sampai 0,17 milidetik pada HC dan SA, dan sekitar 0,08 sampai 0,12 milidetik pada GA. Pada jumlah evaluasi yang mirip (sekitar 20.000), SA (2,4 sampai 2,6 detik) dan GA populasi 100 (1,70 sampai 2,21 detik) berdurasi dalam kisaran yang sama. HC lebih lama (2,8 sampai 4,7 detik) karena evaluasinya lebih banyak, yaitu 21.200 sampai 31.400. GA populasi 200 memakai sekitar 40.000 evaluasi dan berdurasi 3,37 sampai 4,30 detik, mirip HC. Durasi HC juga

bervariasi mengikuti kapan sideways move habis. Seed 2 berjalan 157 iterasi dan memakan 4,695 detik, sedangkan seed 3 berhenti di iterasi 106 dalam 2,810 detik.

Angka durasi bergantung pada perangkat dan beban saat run, jadi selisih kecil antar algoritma tidak perlu dibaca sebagai perbedaan nyata.

2.5.5. Konsistensi Hasil

Tabel 2.5.5.1 Konsistensi Hasil antar Run

Algoritma	Nilai terendah	Nilai tertinggi	Rentang	Simpangan baku
Hill Climbing (sideways)	181	189	8	4,04
Simulated Annealing	187	189	2	1,15
GA, populasi 50, 100 generasi (A)	163	176	13	7,00
GA, populasi 50, 100 generasi (B)	129	162	33	16,62
GA, populasi 100, 100 generasi	173	178	5	2,65
GA, populasi 200, 100 generasi	181	187	6	3,06

SA paling konsisten, dengan rentang 2 dan simpangan baku 1,15. HC berada di tengah dengan rentang 8. GA bergantung pada ukuran populasi: pada populasi 50 simpangan bakunya 7,00 dan 16,62, sedangkan pada populasi 100 dan 200 turun menjadi 2,65 dan 3,06, sejalan dengan pembahasan pada bagian 2.4.4.

Konfigurasi GA yang sama (populasi 50, 100 generasi) dijalankan di skenario A dan B, dan rata-ratanya 168,00 dan 144,33. Selisih 23,67 antar dua set run dari konfigurasi yang sama ini lebih besar daripada selisih antara beberapa konfigurasi GA yang berbeda, jadi pada populasi kecil perbedaan antar konfigurasi perlu ditafsirkan dengan hati-hati.

Hasil HC tampak mengikuti titik awal: run dari state awal −1297 mencapai 189, dari 95 mencapai 184, dan dari −7199 mencapai 181. Pada SA hasil akhirnya hampir sama untuk ketiga titik awal. Dengan tiga run, ini baru indikasi dan bukan bukti.

2.5.6. Pengaruh Objective Berbobot ETA

Objective berbobot ETA mengalikan *Shipping Fee* tiap kendaraan dengan bobot 1 + (ETAmaks − ETA_i) / ETAmaks. Pada demo.json, ETAmaks = 7 (kendaraan D), sehingga D mendapat bobot terendah, yaitu 1,0. Kendaraan F dengan ETA 1 mendapat bobot tertinggi, yaitu 1,857. Bobot kendaraan lain adalah 1,714 untuk C dan J, 1,571 untuk A dan H, 1,429 untuk E dan K, 1,286 untuk B dan I, serta 1,143 untuk G dan L. Penalti pelanggaran tidak diubah, dan parameter kedua algoritma tetap sama dengan percobaan standar (seed 1, 2, 3).

Pembobotan ini mengubah himpunan kendaraan yang optimal. Batas atas knapsack dengan fee berbobot adalah 280,43, dicapai oleh kendaraan A, C, D, E, F, H, I, J, K dengan total berat 118 dan total fee 185. Batas atas fee murni adalah 189, dicapai oleh A, D, E, F, G, H, I, J, K dengan total berat 120. Perbedaannya hanya satu kendaraan: G diganti C. Penggantian ini menurunkan fee sebesar 4, tetapi menaikkan nilai berbobot dari 279,29 menjadi 280,43 karena C lebih mendesak (ETA 2) daripada G (ETA 6). Selisih nilai berbobot antara kedua himpunan hanya 1,14, atau sekitar 0,4%, sehingga pada data ini pembobotan menggeser solusi optimum dengan pengaruh yang relatif kecil.

Pada percobaan dengan objective berbobot, Hill-climbing menghasilkan nilai akhir 275,71, 280,43, dan 275,71 untuk seed 1, 2, dan 3, dengan rata-rata 277,28. Simulated Annealing menghasilkan 275,71, 280,43, dan 280,43, dengan rata-rata 278,86. Seluruh solusi bersifat valid dengan total berat 118. Seed 2 pada kedua algoritma serta seed 3 pada Simulated Annealing berhasil mencapai batas atas 280,43. Seed 1 pada kedua algoritma dan seed 3 pada Hill-climbing berhenti di angka 275,71 (kendaraan A, B, C, E, F, G, H, I, J), yang berjarak 4,71 dari batas atas. Seed 1 juga terbukti paling sulit pada objective fee biasa, di mana Simulated Annealing berhenti di angka 187 dan Hill-climbing di angka 184.

Total Shipping Fee dari solusi berbobot relatif lebih rendah daripada hasil objective biasa. Hill-climbing memperoleh fee 176, 185, dan 176 (rata-rata 179,0), dibandingkan 184, 189, dan 181 (rata-rata 184,67) pada objective biasa. Simulated Annealing memperoleh 176, 185, dan 185 (rata-rata 182,0), dibandingkan 187, 189, dan 189 (rata-rata 188,33). Dengan demikian, objective berbobot mengorbankan sekitar 5 hingga 6 poin fee rata-rata demi memprioritaskan kendaraan yang lebih mendesak.

Perubahan tersebut terlihat jelas pada komposisi kendaraan yang dipilih. Kendaraan C (ETA 2) terangkut pada keenam run berbobot, sedangkan pada objective biasa hanya terangkut pada satu dari enam run (Hill-climbing seed 3).

Kendaraan D (ETA 7, bobot terendah) terangkut pada keenam run objective biasa, tetapi hanya ada pada tiga dari enam run berbobot (Hill-climbing seed 2, Simulated Annealing seed 2 dan 3).

Jika solusi dari objective biasa dinilai ulang dengan bobot ETA, nilainya untuk Hill-climbing adalah 263,14, 279,29, dan 261,00 (rata-rata 267,81), serta untuk Simulated Annealing 277,00, 279,29, dan 279,29 (rata-rata 278,53). Rata-rata objective berbobot Hill-climbing (277,28) lebih tinggi 9,47 daripada nilai evaluasi ulang tersebut, sedangkan Simulated Annealing (278,86) hanya lebih tinggi 0,33. Dengan pengujian tiga seed, selisih pada Hill-climbing ini lebih tepat diartikan sebagai akibat dari jalur pencarian yang berbeda daripada bukti bahwa Hill-climbing lebih diuntungkan oleh pembobotan. Skala nilai berbobot (bobot 1 sampai 2) juga berbeda dari fee biasa, sehingga perbandingan antar-objective sebaiknya menyertakan total fee dan nilai berbobot secara bersamaan seperti di atas.

BAB 3 - KESIMPULAN DAN SARAN

3.1. Kesimpulan

Persoalan penataan kendaraan di kapal untuk memaksimalkan shipping fee berhasil diselesaikan oleh ketiga algoritma local search yang diimplementasikan, yaitu Hill Climbing, Simulated Annealing, dan Genetic Algorithm. Seluruh percobaan pada data uji demo.json menghasilkan susunan kendaraan yang sah terhadap constraint-constraint persoalan dengan nilai objective maksimal sebesar 189. Maka kita anggap 189 ini adalah nilai global optimum untuk test case persoalan ini.

Hill Climbing dengan sideways move menghasilkan rata-rata nilai akhir 184,67 (97,7% dari optimum) dan mencapai global optimum pada satu dari tiga percobaan, dengan durasi rata-rata 3,74 detik yang paling lama di antara variannya. Dari tiga varian bonus yang diuji, random restart memberikan hasil terbaik dengan rata-rata 188,33 (99,6% dari optimum). Steepest ascent justru menghasilkan nilai terendah meskipun waktu eksekusinya paling cepat (0,09 detik).

Simulated Annealing mencatat performa yang paling baik secara keseluruhan dengan rata-rata nilai akhir 188,33, mencapai optimum global pada dua dari tiga percobaan, dan menjadi algoritma paling konsisten. Didapat bahwa dengan menerima langkah yang lebih buruk dengan peluang e^(ΔE/T) efektif mengeluarkan proses pencarian dari local optimum.

Genetic Algorithm menunjukkan performa yang sangat bergantung pada konfigurasi parameter. Jumlah populasi terbukti berpengaruh lebih kuat daripada banyak iterasi. Namun penambahan iterasi dari tidak signifikan menaikkan rata-rata nilai objective. Konfigurasi terbaik dengan populasi 200 dan 100 generasi mencapai rata-rata 184,33. Meskipun begitu belum ada satupun dari percobaan ekseperimen GA dengan konfigurasi yang digunakan yang dapat mencapai global optimum.

Secara umum, mekanisme yang menambah usaha pencarian, baik sideways move, pengulangan dari titik awal baru, maupun populasi yang lebih besar, menghasilkan nilai lebih baik dengan durasi pencarian lebih panjang. Durasi pencarian juga terbukti berbanding lurus dengan jumlah evaluasi objective pada masing-masing algoritma. Secara kedekatan dengan nilai global optimum, konsistensi antar run, dan durasi,

Simulated Annealing lebih unggul untuk persoalan ini, diikuti oleh random restart Hill Climbing dan Genetic Algorithm dengan populasi besar.

3.2. Saran

Untuk pengembangan selanjutnya, disarankan untuk memperbanyak jumlah run pada setiap eksperimen agar kesimpulan mengenai kualitas dan konsistensi algoritma lebih kuat secara statistik, karena ketiga algoritma ini bersifat stokastik dan hasil tiga run masih bisa berbeda jauh pada konfigurasi yang sama. Juga untuk melakukan variasi pada parameter utama masing-masing algoritma, mengingat pada eksperimen ini sebagian besar algoritma hanya diuji dengan satu set parameter sehingga pengaruh parameter terhadap hasil belum dapat dipastikan sepenuhnya. Terakhir, pengujian pada data yang lebih besar dan beragam perlu dilakukan untuk melihat skalabilitas serta ketangguhan ketiga algoritma pada persoalan dengan ruang pencarian yang lebih kompleks.

BAB 4 - PEMBAGIAN TUGAS

1. Gregory William Sutjipto - 18224010
    - Pengerjaan Bonus 1 & 2 (Variant Hill Climbing)
    - Penjalanan demo untuk seluruh varian hill climbing, simulated annealing, dan juga hill climbing
    - Pembuatan dan pengerjaan laporan bagian Bab Pendahuluan, Bab Pembahasan Rancangan Implementasi Umum
2. Johanes Sebastian Siddhadeza - 18224062
    - Pengerjaan Algoritma Hill Climbing dan Simulated Annealing
    - Menjalankan eksperimen pengujian Simulated Annealing dan Hill Climbing
    - Mengerjakan penulisan laporan bagian Algoritma Hill Climbing, Simulated Annealing, & Analisis Perbandingan Masing-Masing Algoritma
3. Emir Muhammad Firassiyan - 18224066
    - Pengerjaan Algoritma Genetic Algorithm
    - Pengerjaan fungsi-fungsi dasar (state, objective, evaluate, visual)
    - Laporan Pengerjaan Tugas Besar bagian algoritma genetic algorithm, kesimpulan & saran, lampiran

BAB 5 - REFERENSI

Deb, K. (2001). Multi-objective optimization using evolutionary algorithms. John Wiley & Sons.

Černý, V. (1985). Thermodynamical approach to the traveling salesman problem: An efficient simulation algorithm. Journal of Optimization Theory and Applications, 45(1), 41-51.

Holland, J. H. (1975). Adaptation in natural and artificial systems. University of Michigan Press.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671-680.

Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equation of state calculations by fast computing machines. The Journal of Chemical Physics, 21(6), 1087-1092.

Russell, S., & Norvig, P. (2020). Artificial intelligence: A modern approach (4th ed.). Pearson.

TutorialsPoint. (n.d.). Genetic algorithms: Survivor selection. https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_survivor_selection.htm

BAB 6 - LAMPIRAN

Github Repository: https://github.com/Joweeverse/IF3070_G29_dai-dai-dai

Surat Pernyataan AI: SURAT PERNYATAAN PENGGUNAAN AI.pdf

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM : Gregory William Sutjipto (18224010), Johanes Sebastian Siddhadeza (18224062), Emir Muhammad Firassiyan (18224066)

Fakultas/Sekolah : STEI-K

Kode Mata Kuliah : IF3070

Nama Mata Kuliah : Dasar Inteligensi Artifisial

Judul Tugas/Proposal : TUGAS BESAR 1 - Penerapan Hill-Climbing, Simulated Annealing, dan Genetic Algorithm untuk Memaksimalkan Shipping Fee pada Penataan Muatan Kapal

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

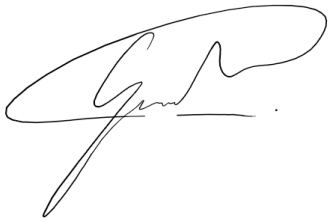
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak*	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	Rendah (membantu pembuatan kerangka)
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	Sedang (brainstorming cara menyelesaikan masalah)
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya/~~Tidak~~*	Rendah (Generate ilustrasi gambar kapal)

Lainnya, Jelaskan: Pemahaman dasar terkait gambaran dasar tugas dan validasi hasil pengerjaan	Ya/~~Tidak~~*	-

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Gregory William Sutjipto
18224010



Johanes Sebastian Siddhadeza
18224062



Emir Muhammad Firassiyan
18224066